



WEST CAPITAL LENDING

---

# CLR Connection Center Complete System Manual

Complete Feature, Admin, and Technical Reference

VERSION 1.0

April 2026

West Capital Lending — Internal Documentation  
Classification: Internal Use Only

# Table of Contents

---

This manual is organized into six parts, 48 chapters. Each chapter is self-contained and cross-referenced where relevant.

## Part I — Overview

*Orientation, roles, and first-run setup.*

|                                         |   |
|-----------------------------------------|---|
| <b>Chapter 1.</b> Introduction          | 3 |
| <b>Chapter 2.</b> Roles and Permissions | 6 |
| <b>Chapter 3.</b> Getting Started       | 9 |

---

## Part II — User-Facing Pages

*A walkthrough of every screen in the application.*

|                                                 |    |
|-------------------------------------------------|----|
| <b>Chapter 4.</b> Dashboard                     | 12 |
| <b>Chapter 5.</b> Script                        | 15 |
| <b>Chapter 6.</b> Assignments                   | 17 |
| <b>Chapter 7.</b> Directory                     | 19 |
| <b>Chapter 8.</b> Call History                  | 21 |
| <b>Chapter 9.</b> EOD Report                    | 24 |
| <b>Chapter 10.</b> Appointments                 | 27 |
| <b>Chapter 11.</b> Stats and Leaderboard        | 29 |
| <b>Chapter 12.</b> Chat                         | 31 |
| <b>Chapter 13.</b> Forum                        | 34 |
| <b>Chapter 14.</b> LO Stats                     | 37 |
| <b>Chapter 15.</b> LO Vacation                  | 39 |
| <b>Chapter 16.</b> Glossary                     | 41 |
| <b>Chapter 17.</b> State Lookup                 | 43 |
| <b>Chapter 18.</b> NMLS License and NMLS Checks | 45 |
| <b>Chapter 19.</b> Reports                      | 47 |

---

|                                     |    |
|-------------------------------------|----|
| <b>Chapter 20.</b> My Report        | 49 |
| <b>Chapter 21.</b> Settings (Admin) | 51 |
| <b>Chapter 22.</b> Team Stats       | 53 |

## Part III — Admin & Integrations

*Administrative workflows, audit, and third-party integrations.*

|                                                |    |
|------------------------------------------------|----|
| <b>Chapter 23.</b> Daily Assignment Generation | 55 |
| <b>Chapter 24.</b> Audit Log                   | 57 |
| <b>Chapter 25.</b> Super Admin Panel           | 59 |
| <b>Chapter 26.</b> Integrations Overview       | 61 |
| <b>Chapter 27.</b> Mojo Integration            | 63 |
| <b>Chapter 28.</b> Bonzo Integration           | 65 |
| <b>Chapter 29.</b> Contact Hub                 | 67 |
| <b>Chapter 30.</b> Webhook Settings            | 69 |

## Part IV — Technical Architecture

*Stack, schema, API surface, and cross-cutting concerns.*

|                                                |     |
|------------------------------------------------|-----|
| <b>Chapter 31.</b> Frontend Stack              | 71  |
| <b>Chapter 32.</b> Backend Stack               | 74  |
| <b>Chapter 33.</b> Database Schema             | 77  |
| <b>Chapter 34.</b> API Reference               | 80  |
| <b>Chapter 35.</b> Authentication and Sessions | 99  |
| <b>Chapter 36.</b> Progressive Web App         | 101 |
| <b>Chapter 37.</b> Timezone Handling           | 103 |
| <b>Chapter 38.</b> Realtime Features           | 105 |
| <b>Chapter 39.</b> Web Push Notifications      | 107 |

## Part V — Operations

*Deployment, monitoring, backups, and incident response.*

|                                          |     |
|------------------------------------------|-----|
| <b>Chapter 40.</b> Deployment to Railway | 109 |
|------------------------------------------|-----|

|                                                   |     |
|---------------------------------------------------|-----|
| <b>Chapter 41.</b> Cache Busting and Redeployment | 112 |
| <b>Chapter 42.</b> Monitoring and Health Checks   | 114 |
| <b>Chapter 43.</b> Database Backup and Volume     | 116 |
| <b>Chapter 44.</b> Incident Response              | 118 |

## Part VI — Reference

*Changelog, glossary, FAQ, and support contacts.*

|                                           |     |
|-------------------------------------------|-----|
| <b>Chapter 45.</b> Changelog (April 2026) | 121 |
| <b>Chapter 46.</b> Glossary of Terms      | 124 |
| <b>Chapter 47.</b> FAQ                    | 127 |
| <b>Chapter 48.</b> Contact and Support    | 131 |

# Introduction

## In this chapter

A tour of the CLR Connection Center: what it is, who uses it, and why it exists.

## Purpose of this manual

This document is the complete reference for the CLR Connection Center application — every page, every role, every admin toggle, every integration, every API endpoint, and the operational playbook that keeps the system healthy. It is intended for three audiences: CLRs learning the product, admins configuring it, and engineers operating or extending it. Each chapter is self-contained and cross-referenced where relevant.

## What is CLR Connection Center?

CLR Connection Center is West Capital Lending's internal operations platform for Call Lead Representatives (CLRs) — the team members who take inbound mortgage leads, qualify them, and hand them off to Loan Officers (LOs). It sits between the dialer (Mojo) and the CRM (Bonzo) and provides the surface where CLRs do their day-to-day work: log outcomes, submit end-of-day reports, set appointments, and coordinate with the LO team.

## Why the app exists

Before CLR Connection Center, the workflow was a patchwork of spreadsheets, chat threads, email reports, and tribal knowledge. Tracking a CLR's weekly performance required reconciling Mojo call logs with Bonzo appointment records by hand. Missed handoffs, mis-assigned LOs, and late reports were common. The app consolidates all of this into a single surface with integrity built in: every outcome has a timestamp, every appointment has an audit trail, every assignment has a reason code.

## Guiding principles

- Prefer structured data over free text — but always allow free text as an escape hatch.
- Keep the CLR's happy path to under 3 clicks per outcome.
- Admins should never need to write SQL to understand team performance.
- Every sensitive action is audit-logged. Every auditable record is immutable after the day of creation.
- Integrations are opt-in and survive the loss of any single external system.

## How to read this manual

The manual is organized into six parts. Part I (this part) is orientation. Part II walks through every user-facing page the CLR interacts with. Part III covers admin and integration surfaces. Part IV is the technical architecture — frontend, backend, schema, API, real-time, PWA, timezone handling, push. Part V covers operations — deployment, monitoring, backups, incident response. Part VI is reference — changelog, glossary, FAQ, and support.

## Conventions

| Convention               | Meaning                                                                    |
|--------------------------|----------------------------------------------------------------------------|
| Monospaced text          | A literal value: endpoint path, environment variable, table name.          |
| Italics                  | A concept being introduced or emphasized.                                  |
| Callout boxes            | Warnings, tips, and side notes that can be safely skipped on a first read. |
| Chapter cross-references | "See Chapter X" links to the chapter number listed in the TOC.             |

## Version

This is version 1.0 of the manual, dated April 2026. It reflects the state of the application at that date. The changelog in Chapter 45 tracks significant feature changes since the initial launch. This manual is a living document — file a pull request against the documentation source to suggest an edit.

## Related chapters

- **Chapter 2** — Roles and Permissions
- **Chapter 3** — Getting Started
- **Chapter 4** — Dashboard

# Roles and Permissions

## In this chapter

How the four roles map to capabilities, plus the reasoning behind least-privilege.

## Role model

CLR Connection Center has a flat, role-based permission model. Every authenticated user holds exactly one primary role; SuperAdmin is an additional flag layered on top of the Admin role. There are no per-feature permissions — the role determines what pages and endpoints are accessible.

## The roles

| Role       | Who                 | What they can do                                                                                                            |
|------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------|
| CLR        | Call Lead Reps.     | Log outcomes, submit EODs, set appointments, view their own data.                                                           |
| LO         | Loan Officers.      | View transfers routed to them, accept/decline appointments, view their LO Stats profile.                                    |
| Manager    | Team leads.         | Everything a CLR can do plus Team Stats, Reports for their direct reports.                                                  |
| Admin      | Ops admins.         | Everything Manager can do plus Settings, Integrations, Webhook, Daily Assignment overrides, NMLS overrides, Audit Log read. |
| SuperAdmin | Org owner + backup. | Everything Admin can do plus user role management, impersonation, feature flags, emergency lockout.                         |

## Role assignment

New users are created with the CLR role by default. SuperAdmins promote users as needed via **Super Admin Users**. Demotion is also a SuperAdmin-only action. All role changes are audit-logged with before/after snapshots.

## Multi-role users

A user holds exactly one primary role at a time. The SuperAdmin flag is orthogonal: a user with primary role Admin can also carry SuperAdmin. When a user wears multiple hats — e.g., a manager who is also an LO — the more privileged role is used and the app surfaces secondary information where relevant (the user's LO Stats row is still visible in the LO dashboard, for instance).

## What a new CLR sees

On first login, a CLR lands on the Dashboard. The sidebar shows: Dashboard, Script, Assignments, Directory, Call History, EOD Report, Appointments, Stats, Chat, Forum, LO Stats, LO Vacation, Glossary, State Lookup, NMLS, Settings. Admin-only items are hidden.

## What an admin sees

An admin sees everything a CLR sees plus: Reports, Team Stats, Audit Log, Daily Assignment, and additional tabs within Settings. The Admin sidebar section is visually distinct with a gold accent on the label.

## Permission failures

When a user attempts to access a route their role does not permit, the backend returns 403 forbidden with a structured error body. The frontend catches this and displays a friendly page: "This area is for admins only. Ask your admin if you need access." Direct API calls get the raw 403.

## Related chapters

- **Chapter 1** — Introduction
- **Chapter 21** — Settings (Admin)
- **Chapter 24** — Audit Log
- **Chapter 25** — Super Admin Panel

# Getting Started

## In this chapter

First login, profile setup, PWA install, and a verification checklist.

## Overview

This chapter is the fast-path checklist for new users — how to log in, set up your profile, and complete your first shift. It is intentionally short; more detailed chapters follow for each feature.

## Before your first shift

- Receive your invite email from your admin. The invite link is valid for 7 days.
- Click the link, set your password (12-character minimum, not pwned), complete the profile form.
- Add your timezone if it differs from the default (America/Los\_Angeles).
- Upload an avatar — helps teammates recognize you in chat and directory.
- Enable push notifications when prompted (optional but recommended).
- Install the PWA on desktop or phone (optional but reduces distraction).

## Your first day

- Open the app. Default landing is the Dashboard.
- Check your Assignments — which LOs you're paired with today.
- Open the Script page in a second tab; keep it handy during calls.
- When the first call comes in through Mojo, it auto-appears in the outcome form draft.
- As the call progresses, select the appropriate script node.
- When the call ends, log the outcome: transfer, appointment, callback, fell-through, future, no-answer.
- At the end of the day, submit your EOD with counts and a short narrative.

## Getting help

- Forum (Chapter 13) for durable questions.
- Chat (Chapter 12) for right-now help.
- Glossary (Chapter 16) for unfamiliar terms.
- Your admin for anything blocking.
- This manual for detail.

## Ramp-up timeline

| Week     | Expectation                                                                    |
|----------|--------------------------------------------------------------------------------|
| Week 1   | Shadow a senior CLR. Observe without driving.                                  |
| Week 2   | Take calls with a senior on the line. Log outcomes yourself.                   |
| Week 3   | Solo calls, with script adherence. Daily check-in with manager.                |
| Week 4   | Full schedule. Weekly 1:1 with manager to debrief.                             |
| Month 2  | Stats compared to the team baseline. Targeted coaching.                        |
| Month 3+ | Full contributor. Considered for niche assignments (e.g., specialty products). |

## Related chapters

- **Chapter 1** — Introduction
- **Chapter 36** — Progressive Web App
- **Chapter 21** — Settings (Admin)

# Dashboard

## In this chapter

The landing page: KPI cards, weekly goals, the activity trend chart, and recent activity.

## Overview

The Dashboard is the CLR's home page. It opens to a personalized summary of the day so far: calls logged, outcomes by type, current streak, goal progress, and a live activity feed from the rest of the team. It is designed to answer one question at a glance — *am I on pace?*

## Layout

| Region      | Contents                                                                                        |
|-------------|-------------------------------------------------------------------------------------------------|
| Top row     | KPI tiles — calls today, transfers today, appointments today, fell-throughs today.              |
| Middle row  | Goal progress bars — daily, weekly, monthly. Color-coded (green 100%, yellow 70–99%, red <70%). |
| Left panel  | Recent outcomes — the last 10 you logged, chronological, click to edit.                         |
| Right panel | Team activity feed — live stream of outcomes being logged by teammates.                         |
| Bottom row  | Action shortcuts — "Log outcome", "Open script", "Open chat", "View my report".                 |

## KPI computation

Each tile is a server-computed number from GET /api/dashboard/summary. The endpoint accepts an optional date parameter for past-day review. The client polls it every 60 seconds while the tab is visible to keep counts fresh.

## Goal bars

Goals are set per-user by an admin in **Settings Goals**. If no goal is set for a metric, the bar is hidden. Daily goals reset at 00:00 America/Los\_Angeles; weekly goals reset Monday 00:00; monthly goals reset the first day of the month 00:00. All boundaries are business-timezone-aware.

## Activity feed

The team activity feed is powered by the same WebSocket channel that drives Chat (Chapter 38). When any teammate logs an outcome, an event is broadcast and appended to the feed. The feed holds the last 50 items in memory and fades older items to muted gray.

## Empty states

On a fresh day with no activity yet, the dashboard shows a lightweight welcome card: "Good morning. No calls yet today. Hit the script or open assignments to get started." It is not meant to be nagging — it is meant to be a gentle nudge.

## Keyboard shortcuts

| Key | Action                            |
|-----|-----------------------------------|
| g d | Go to Dashboard.                  |
| n o | New outcome (opens outcome form). |
| n e | New EOD (opens EOD form).         |
| ?   | Show full keyboard shortcuts.     |

## Related chapters

- **Chapter 5** — Script
- **Chapter 6** — Assignments
- **Chapter 11** — Stats and Leaderboard
- **Chapter 38** — Realtime Features

# Script

## In this chapter

The calling script with live placeholders, Start Call mode, and the admin's branching editor.

## Overview

Script is the CLR's live cue-card: a branching flow of lines to use during a call, shaped by the borrower's responses. It is a training tool for new hires and a safety net for experienced CLRs on unfamiliar products.

## Script structure

- Each script is a directed graph of nodes.
- A node contains the line(s) to deliver, plus 0–6 response buttons.
- Clicking a response button navigates to the next node and logs the response.
- Terminal nodes link to outcome types: "Transfer", "Appointment", "Callback", "Fell Through", etc.
- Scripts are versioned: edits create new versions; old versions remain viewable for audit.

## Editing a script

Admins edit scripts in **Settings Scripts**. The editor is a visual node canvas: drag to create, connect by drawing lines, double-click to edit content. A save publishes a new version; a rollback reverts to any prior version.

## Script telemetry

The app logs which node a CLR was on when an outcome occurred. Over time this reveals which branches convert and which dead-end. GET /api/scripts/:id/telemetry returns a heatmap of node visits and conversion per response button.

## API

- GET /api/scripts — list.
- GET /api/scripts/:id — full graph.
- POST /api/scripts — create.
- PUT /api/scripts/:id — publish new version.
- POST /api/scripts/:id/responses — record a response during a live call.

## Etiquette for CLRs

The script is a tool, not a crutch. Read naturally, deviate when appropriate, and circle back. If a borrower's objection is not represented, log it via the "Suggest" button at the bottom of every node — your suggestion goes to admins for the next revision.

## Related chapters

- **Chapter 6** — Assignments
- **Chapter 8** — Call History
- **Chapter 34** — API Reference

# Assignments

**In this chapter**

Today's assigned LOs, daily generation and lock rules, and per-row admin overrides.

## Overview

Assignments are the list of LOs a CLR is expected to work with today. The assignment engine (Chapter 23) generates them nightly based on LO availability, vacation, state coverage, and round-robin fairness.

## What a CLR sees

| Column           | Detail                                                                         |
|------------------|--------------------------------------------------------------------------------|
| LO name          | Bold if this is your primary LO for the day.                                   |
| State coverage   | States this LO is licensed for, colored green/yellow/red based on NMLS status. |
| Typical response | Avg time from transfer to first contact.                                       |
| Today's activity | # of calls and transfers routed to this LO so far today.                       |
| Notes            | Admin notes for this LO (e.g., "out until 3 PM, use Ana as backup").           |

## Reassignment

If an LO becomes unavailable mid-day (no-shows a transfer, for instance), the CLR can request a reassignment: click the refresh icon next to the LO row. The engine rebalances on the fly. All reassignments are audit-logged with reason code.

## API

- GET /api/assignments/today — the current day's assignments for the logged-in CLR.
- GET /api/assignments/history — prior days, defaults to 14-day window.
- POST /api/assignments/reassign — request a mid-day rebalance.

## Fairness model

Assignments aim for rough fairness: over a 30-day window, each CLR should see each LO about equally often. Exact fairness is impossible because LOs are licensed in different states and a CLR's inbound calls come from uneven states. The engine uses a weighted round-robin and includes a monthly imbalance report for admins to adjust.

## Related chapters

- **Chapter 7** — Directory
- **Chapter 23** — Daily Assignment Generation
- **Chapter 21** — Settings (Admin)

# Directory

## In this chapter

The full LO directory — contact fields, NMLS IDs, and the CSV bulk import.

## Overview

Directory is the searchable roster of all loan officers, CLRs, and admins. It is the fastest path to look up someone's contact info, state coverage, NMLS number, or current availability.

## Filters

- Role (LO, CLR, Admin, SuperAdmin).
- State (shows anyone licensed in that state).
- Availability (on-shift, on-vacation, offline).
- Tag (admin-applied labels like "senior", "bilingual", "jumbo").

## Row detail

| Field         | Detail                                                           |
|---------------|------------------------------------------------------------------|
| Avatar & name | Clickable; opens full profile side panel.                        |
| Role badge    | Color-coded: LO green, CLR blue, Admin purple, SuperAdmin gold.  |
| NMLS #        | With status chip (Active / Expiring / Expired).                  |
| States        | Comma-separated abbreviations, truncated to 3 with a +N tooltip. |
| Phone / email | Click to copy.                                                   |
| Chat          | Shortcut to open a DM channel.                                   |

## API

- GET /api/directory — full roster.
- GET /api/directory/:id — one user's profile with extended detail.

## Access

All authenticated users can open the directory. Profile fields marked private (salary, birthday) are only visible to admins.

## Related chapters

- **Chapter 14** — LO Stats
- **Chapter 15** — LO Vacation
- **Chapter 18** — NMLS License and NMLS Checks
- **Chapter 29** — Contact Hub

# Call History

## In this chapter

Outcome log: the six outcome types, filtering, editing after the fact, and CSV export.

## Overview

Call History is the CLR's personal log of every call they've taken. It is a detailed time-ordered feed with filters, useful for retrospective analysis and for looking up a prior interaction with a specific borrower.

## Columns

| Column      | Detail                                                            |
|-------------|-------------------------------------------------------------------|
| Time        | Call start time, local to user.                                   |
| Duration    | HH:MM:SS.                                                         |
| Borrower    | Name and phone. Click to open Contact Hub.                        |
| State       | Borrower state.                                                   |
| Outcome     | Transfer, appointment, callback, fell-through, future, no-answer. |
| LO          | If transferred, which LO.                                         |
| Script node | Terminal node in the script at time of outcome.                   |
| Notes       | First 80 chars of CLR notes; hover to see full.                   |

## Filtering

Time range (today, this week, this month, custom); outcome (multi-select); state (multi-select); LO (multi-select); free-text search across borrower name and notes. Filters combine with AND; state and outcome within a filter combine with OR.

## Export

Export button produces a CSV of the filtered rows for use in spreadsheets. The export respects the current filter set. The file is timestamped and includes a header row.

## API

- GET /api/calls?from=&to;=&outcome;=&state;=&lo;=&q;= — paginated.
- GET /api/calls/:id — full detail.
- GET /api/calls/export?... — CSV stream.

## Related chapters

- **Chapter 9** — EOD Report
- **Chapter 10** — Appointments

- **Chapter 11** — Stats and Leaderboard

# EOD Report

**In this chapter**

End-of-Day reporting, the print-sheet PDF export, and admin history view.

## Overview

The End-of-Day (EOD) report is a short structured form each CLR submits at the end of their shift. It captures the day's activity totals, a free-form narrative reflection, and any follow-up items for tomorrow. Taken together, EODs form the backbone of weekly reports (Chapter 19).

## Form fields

| Field           | Detail                                                                              |
|-----------------|-------------------------------------------------------------------------------------|
| Date            | Auto-filled to today. Editable by admins to backfill.                               |
| Calls           | Number of calls handled. Pre-filled from logged outcomes; editable if counts drift. |
| Transfers       | Successful transfers to an LO.                                                      |
| Appointments    | Appointments set.                                                                   |
| Fell-throughs   | Deals that started and died mid-call.                                               |
| Callbacks set   | Follow-ups scheduled with the borrower.                                             |
| Narrative       | What went well, what didn't, anything the team should know. 1-3 paragraphs.         |
| Follow-ups      | Named items for tomorrow — carries into tomorrow's dashboard.                       |
| Mood (optional) | Emoji picker — helps managers spot burnout signals.                                 |

## Submission rules

- EOD is one per user per day. Submitting a second time updates the first.
- Cutoff is 11:59 PM local. After that, submissions are flagged as late.
- Admins can submit on a CLR's behalf (e.g., if the CLR is sick and forgot).
- Submitting triggers a notification to the CLR's manager.
- Three consecutive missed EODs flag an alert on the Team Stats watchlist.

## Activities detail

Each EOD can include one or more activity lines with finer granularity than the top-level counts — e.g., "Coached Jamie on the objection script for 20 min" or "Tested the new dashboard for bugs". These live in the `eod_report_activities` table and surface in reports under "Non-call time".

## API

- GET `/api/eod-reports` — list; filter by date, user.
- GET `/api/eod-reports/:id` — detail including activities.

- POST /api/eod-reports — submit or update.
- DELETE /api/eod-reports/:id — admin-only.

## Best practice

Keep the narrative short and specific. "Good day" tells nobody anything; "Rejected two transfers because LO Maria was on vacation and nobody else covers OR" is an action item for the admin. The narrative is the most valuable field in the form — use it.

## Related chapters

- **Chapter 8** — Call History
- **Chapter 10** — Appointments
- **Chapter 19** — Reports
- **Chapter 20** — My Report

# Appointments

**In this chapter**

Appointments, callbacks, deferrals, and the three-day badge rule.

## Overview

Appointments tracks every appointment set during calls — the borrower, the LO, the date/time, and the current status. It is the bridge between the CLR's top-of-funnel work and the downstream deal team.

## Columns

| Column   | Detail                                                                 |
|----------|------------------------------------------------------------------------|
| When     | Appointment date and time, localized.                                  |
| Borrower | Name, phone, state.                                                    |
| LO       | Assigned LO.                                                           |
| CLR      | The CLR who set it.                                                    |
| Status   | Scheduled, Confirmed, Held, No-showed, Rescheduled, Cancelled, Closed. |
| Notes    | Context from the setting call.                                         |
| Source   | How the appointment was set — script node, manual add.                 |

## Status transitions

- Scheduled Confirmed: an LO acknowledges the appointment.
- Confirmed Held: the appointment happened.
- Confirmed No-showed: the borrower didn't show.
- Any Rescheduled: new time set; old record archived.
- Any Cancelled: explicit cancellation.
- Held Closed: the loan closed. Only reachable via Bonzo sync.

## Notifications

When an appointment is created, the assigned LO gets a chat DM and a push notification. 24 hours before the appointment, a reminder fires to both LO and CLR. No-showed appointments trigger a reflexive EOD note asking the CLR to document the outcome.

## API

- GET /api/appointments — filterable list.
- GET /api/appointments/:id
- POST /api/appointments
- PATCH /api/appointments/:id
- POST /api/appointments/:id/transition — explicit status change.

## Related chapters

- **Chapter 8** — Call History
- **Chapter 9** — EOD Report
- **Chapter 39** — Web Push Notifications

# Stats and Leaderboard

**In this chapter**

Team leaderboard, sortable columns, the 1D/1W/1M/All toggle.

## Overview

The Stats and Leaderboard page is a friendly competition surface — each CLR's weekly numbers rendered as a ranked list. It nudges healthy motivation without encouraging gaming: the metrics are quality-weighted, not raw volume.

## Metrics

| Metric        | Definition                                                                 |
|---------------|----------------------------------------------------------------------------|
| Transfers     | Count of successful transfers in the week.                                 |
| Appointments  | Count of appointments set that were subsequently Held.                     |
| Conversion    | Transfers / Calls, rendered as a percent.                                  |
| Quality score | A composite: transfers weighted 1.0, appointments 1.5, fell-throughs -0.5. |
| Rank          | Position within the team for the week.                                     |
| Trend         | Arrow indicating whether this week is better or worse than last week.      |

## Backing endpoint

GET /api/leaderboard?week=YYYY-WW returns the ranked array. Default week is the current reporting week. Admins can query any week; CLRs can query only their own team's history.

## Privacy

The leaderboard is visible to everyone on the team by default. A per-user opt-out hides a CLR's name and renders them as "Teammate" on the public view. Admins still see real names.

## Anti-gaming

The quality score penalizes fell-throughs and unreturned callbacks, which discourages picking up the phone, flashing a transfer, and hanging up. Suspicious patterns — e.g., a high transfer count paired with a high fell-through count — trigger a manager alert for review.

## Related chapters

- **Chapter 4** — Dashboard
- **Chapter 22** — Team Stats
- **Chapter 19** — Reports

# Chat

## In this chapter

Real-time team chat — polling mechanics and admin moderation.

## Overview

Chat is the app's built-in real-time team messenger. It is room-based: every channel has a name, a list of members, and a running log of messages. It is not a replacement for Slack — it is the fastest path for ops-critical, in-the-moment team communication ("anyone free for a transfer?", "system alert", "hot lead just came in").

## Channels and membership

- Admins can create, rename, and archive channels.
- Channels can be open (visible to everyone) or invite-only.
- A #general channel exists by default and cannot be archived.
- Direct messages are a special case of a two-member channel.
- Members receive browser and push notifications when mentioned by @name.

## Message features

| Feature          | Detail                                                                  |
|------------------|-------------------------------------------------------------------------|
| Send text        | Enter sends, Shift+Enter inserts newline.                               |
| @mentions        | @name highlights and notifies that user.                                |
| Edit             | Pencil icon within 5 minutes of send (admin can edit any).              |
| Delete           | Trash icon — soft-delete, message replaced with "message removed".      |
| Reactions        | Emoji reactions via hover toolbar.                                      |
| Typing indicator | "X is typing..." appears when another member is composing.              |
| Unread badge     | Side nav shows unread count per channel; clears on view.                |
| Read receipts    | Members who have opened the channel since your message show tick marks. |

## Real-time plumbing

Chat uses a WebSocket channel (Chapter 38) for push updates. On connection loss the client falls back to polling every 10 seconds until the socket reconnects. Messages are persisted to the chat\_messages table and replayed on reconnection so nothing is lost.

## API endpoints

- GET /api/chat/channels — list channels this user is a member of.
- POST /api/chat/channels — create a channel (admin only).
- GET /api/chat/channels/:id/messages?limit=50&before=ts — paginate history.
- POST /api/chat/channels/:id/messages — post a message.
- PATCH /api/chat/messages/:id — edit.
- DELETE /api/chat/messages/:id — soft-delete.
- POST /api/chat/messages/:id/react — add or remove a reaction.

## Etiquette

Chat is for right-now coordination. Anything that needs to live past today — customer notes, decisions, follow-ups — belongs in an outcome record or an EOD. If you find yourself writing a paragraph in chat, open the forum or file a record instead.

## Moderation

Admins see a red Moderation tab when they hover a message. From there they can delete, edit, or mute a user. Muted users can still read but cannot post for the configured duration. All moderation actions generate audit-log entries (Chapter 24).

## Related chapters

- **Chapter 13** — Forum
- **Chapter 38** — Realtime Features
- **Chapter 39** — Web Push Notifications

# Forum

## In this chapter

Forum posts, threaded answers, upvotes, and accepted answers.

## Overview

Forum is a long-form Q&A; surface: one topic per thread, answers voted on, and pinned threads at the top. It is the right place for anything that needs to be findable weeks later ("how do I handle a borrower in CA with an expired NMLS..."). Chat is synchronous, Forum is asynchronous.

## Thread lifecycle

- A user posts a question with a title, body (Markdown allowed), and tags.
- Other users answer; one answer can be marked as Accepted by the OP or an admin.
- Members upvote or downvote answers; the answer with the highest score floats to the top.
- Admins can pin threads (stuck to top of list) and lock threads (no new replies).
- Archived threads are hidden from the default list but searchable.

## Suggested tag taxonomy

- `#script` — questions about the sales script.
- `#state-X` — state-specific questions (e.g., `#state-CA`).
- `#lo-X` — LO-specific questions (use initials for privacy).
- `#product` — loan product questions.
- `#ops` — tooling, Mojo, Bonzo, app bugs.
- `#compliance` — NMLS, licensing, legal.

## Search

The forum supports full-text search via `GET /api/forum/search?q=`. It searches title + body + answers and ranks by relevance and recency. A global search shortcut (`Ctrl+K`) surfaces the top 5 matching threads inline alongside other results.

## API endpoints

- `GET /api/forum/posts` — list (with pagination).
- `POST /api/forum/posts` — create.
- `GET /api/forum/posts/:id` — detail including answers.
- `POST /api/forum/posts/:id/answers` — answer a thread.
- `POST /api/forum/answers/:id/vote` — +1 / -1.
- `POST /api/forum/answers/:id/accept` — mark accepted.
- `POST /api/forum/posts/:id/pin` (admin) and `/lock`.

## Moderation and etiquette

Expected behavior is close to Stack Overflow: one question per thread, answers addressing that question, and votes reflecting quality rather than opinion. Down-voting should include a comment explaining why. Admins may edit titles for clarity, move threads to the right category, and close duplicates with a link to the canonical thread.

## When to use which

| Question                                           | Where                    |
|----------------------------------------------------|--------------------------|
| "Hey can anyone handle this call?"                 | Chat.                    |
| "What do I say when a borrower objects to rate?"   | Forum — durable answer.  |
| "Is the system down?"                              | Chat #general + @admin.  |
| "Best way to record a callback for a future date?" | Forum.                   |
| "LO X is not responding."                          | Chat DM to your manager. |

## Related chapters

- **Chapter 12** — Chat
- **Chapter 39** — Web Push Notifications
- **Chapter 48** — Contact and Support

# LO Stats

## In this chapter

Per-LO performance analytics.

## Overview

LO Stats shows per-loan-officer performance — not the CLR's activity but the LOs' outcomes once deals reach them. It is the bridge between the CLR's top-of-funnel work and the downstream closing team.

## Metrics per LO

| Metric            | Definition                                                    |
|-------------------|---------------------------------------------------------------|
| Calls received    | Number of transfers routed to this LO in the selected period. |
| Appointments set  | Appointments that originated from a CLR transfer to this LO.  |
| Close rate        | Closed loans / appointments set — proxy for LO effectiveness. |
| Avg response time | Time from transfer accepted to first LO outreach.             |
| Availability rate | % of time the LO was accepting transfers during the period.   |
| Vacation days     | Scheduled out-of-office days (Chapter 15).                    |

## Backing endpoint

GET /api/lo-stats?from=YYYY-MM-DD&to=YYYY-MM-DD returns an array of LO stat records. The period defaults to the current reporting week.

## Filtering and sorting

Clickable column headers sort ascending/descending. A search field filters by LO name or NMLS. An "Active only" toggle hides LOs on vacation. Results export to CSV via the export button.

## Read carefully

LO Stats is a diagnostic, not a ranking. A low close rate might mean poor leads from CLRs, not a weak LO. Cross-reference with **Team Stats** (Chapter 22) and the CLR-side leaderboard before drawing conclusions.

## Related chapters

- **Chapter 7** — Directory
- **Chapter 15** — LO Vacation
- **Chapter 11** — Stats and Leaderboard

# LO Vacation

**In this chapter**

LO availability and vacation, and how it interacts with assignment generation.

## Overview

LO Vacation is the scheduling surface admins use to mark LOs as unavailable. When an LO is on vacation, their row is grayed out in the Directory, the assignment engine skips them, and Team Stats adjusts for their absence when computing averages.

## Fields

| Field             | Detail                                                    |
|-------------------|-----------------------------------------------------------|
| LO                | The loan officer. Dropdown from the directory.            |
| Start date        | First unavailable day (inclusive). Timezone: app default. |
| End date          | Last unavailable day (inclusive). Blank means indefinite. |
| Reason (optional) | Free text — "PTO", "training", "medical". Internal only.  |
| Notify team       | Checkbox — if set, posts a notice to #general on save.    |

## Automation

The assignment logic (Chapter 23) runs nightly. If an LO's vacation window includes tomorrow, they are omitted from tomorrow's assignment slots. When their vacation ends, they are automatically brought back into rotation.

## Edge cases

- Editing a vacation record mid-window adjusts the rest of the window, not history.
- Deleting a record immediately returns the LO to available status.
- Overlapping vacations are allowed (e.g., extended + appended) and the union is used.
- A half-day can be represented by adding a note — the granularity is day-level.

## API

- GET /api/lo-vacations
- POST /api/lo-vacations
- PATCH /api/lo-vacations/:id
- DELETE /api/lo-vacations/:id

## Related chapters

- **Chapter 7** — Directory
- **Chapter 14** — LO Stats
- **Chapter 23** — Daily Assignment Generation

# Glossary

## In this chapter

The CLR-facing glossary of mortgage and operational terms.

## Overview

Glossary is an admin-editable dictionary of mortgage-industry and WCL-specific terms. CLRs use it during calls when a borrower uses unfamiliar jargon, and new hires use it to ramp up.

## Entry shape

| Field       | Detail                                                         |
|-------------|----------------------------------------------------------------|
| Term        | The word or phrase.                                            |
| Definition  | A plain-English explanation (no jargon inside the definition). |
| Example     | A sentence showing the term in context.                        |
| Category    | e.g., Mortgage, Legal, Product, WCL-internal.                  |
| Last edited | Audit timestamp and editor name.                               |

## Search

The search field is case-insensitive, matches prefixes and substrings, and highlights matches in the result list. Ctrl+K from anywhere in the app opens a global search which includes glossary hits.

## API endpoints

- GET /api/glossary — full list.
- POST /api/glossary — add a term (admin only).
- PATCH /api/glossary/:id — edit a term.
- DELETE /api/glossary/:id — remove a term.

## Seeded entries

The app ships with ~30 seeded terms covering mortgage fundamentals (APR, LTV, PITI), loan types (FHA, VA, conventional, jumbo), and WCL-specific shorthand (transfer, fell-through, callback, 3Bs, CLR, LO). Admins expand as needed.

## Best practice

Keep definitions short: one or two sentences. If a concept needs more than that, link out to a forum thread or knowledge-base article instead. The glossary is a quick-glance reference, not a training manual.

## Related chapters

- **Chapter 46** — Glossary of Terms
- **Chapter 21** — Settings (Admin)

# State Lookup

**In this chapter**

Quick reference for state-specific regulatory detail.

## Overview

Quick reference for state-specific information relevant to CLRs: state name, NMLS requirements, any known special rules, and notes. Select a state from the dropdown to see the detail card.

## Fields on each state card

| Field                | What it contains                                                                          |
|----------------------|-------------------------------------------------------------------------------------------|
| State name           | Full state name and abbreviation.                                                         |
| NMLS availability    | Whether the state is licensed for WCL's primary products.                                 |
| Special rules        | State-specific regulatory notes — e.g., waiting periods, rate caps, required disclosures. |
| Operating hours note | Time-zone-adjusted calling window if different from defaults.                             |
| Last reviewed        | Timestamp — when an admin last updated this state's entry.                                |

## Access

|               |                                                              |
|---------------|--------------------------------------------------------------|
| <b>Route</b>  | /state-lookup                                                |
| <b>Access</b> | All authenticated users.                                     |
| <b>Source</b> | Seeded data file plus admin edits persisted in the database. |

## Use case

When a borrower mentions their state and you need a quick sanity check on licensing or state-specific loan program availability before you commit to a transfer, this page is faster than leaving the app to search externally. Bookmark it on a second monitor if you work across multiple states daily.

## Keeping the data current

The state data is fundamentally a reference lookup, not a live feed. An admin should review the entries quarterly — state compliance and product availability move slowly, but disclosures and rate caps change more often. A stale entry is worse than no entry, so keep the Last Reviewed timestamp visible and audit it regularly.

## Related chapters

- **Chapter 18** — NMLS License and NMLS Checks

# NMLS License and NMLS Checks

**In this chapter**  
NMLS license status and the pending-check admin queue.

## Overview

NMLS (Nationwide Multistate Licensing System) is the federal registry for mortgage loan originators. Every LO in the app has an NMLS number and a list of states where they are licensed. The NMLS page verifies status daily and surfaces expirations before they become problems.

## What the page shows

| Column        | Detail                                        |
|---------------|-----------------------------------------------|
| LO            | Name, avatar.                                 |
| NMLS #        | Number.                                       |
| Primary state | LO's home state for licensing.                |
| All states    | Comma-separated list of licensed states.      |
| Status        | Active, Expiring (30 days), Expired, Unknown. |
| Last checked  | Timestamp of last daily verification.         |

## How checks run

A daily cron job hits the NMLS consumer-access endpoint for each LO's number and parses the status. Results are cached in the `nmls_checks` table. Failed lookups fall back to the previous known status with a warning flag.

## Alerts

- 30 days before expiration: a chat DM and email to the LO and the admin.
- 7 days before expiration: escalation to the admin and the LO's manager.
- On expiration: the LO is flagged as Unlicensed and the assignment engine stops routing calls to them for affected states.
- Failed lookups persisting more than 48 hours: alert to ops.

## Manual override

An admin can manually set an LO's status when the automated lookup is wrong (it happens). Manual overrides expire after 30 days to force a re-verification. All overrides are audit-logged.

## API

- GET `/api/nmls` — current status for all LOs.
- POST `/api/nmls/:loId/recheck` — force an immediate lookup.
- PATCH `/api/nmls/:loId` — manual status override.

## Related chapters

- **Chapter 7** — Directory
- **Chapter 15** — LO Vacation
- **Chapter 24** — Audit Log

# Reports

## In this chapter

Weekly team reports with narrative from outcomes and EOD.

## Overview

Reports is the admin's view of weekly rollups. Pick a week; see each CLR's calls, transfers, appointments, fell-throughs, callback breakdown, and narrative notes culled from their outcomes and EOD reports.

## Data sources

- GET /api/reporting-period — the list of reportable weeks with start/end dates.
- GET /api/leaderboard — aggregated per-CLR totals for the range.
- GET /api/outcomes — narrative notes culled from outcome records.
- GET /api/eod-reports — EOD-level narrative.
- GET /api/analytics/history — trend data for in-report charts.

## What a week looks like

| Section       | Contents                                                                         |
|---------------|----------------------------------------------------------------------------------|
| Team totals   | Calls, transfers, appointments, fell-throughs, callbacks. Week over week delta.  |
| Per-CLR rows  | One row per CLR, same columns as leaderboard. Sortable by any metric.            |
| EOD narrative | Free-form notes pulled from the week's submitted EOD reports.                    |
| Outcome notes | Samples of notes the CLRs entered on individual outcomes (redacted for brevity). |
| Chart         | A small activity trend chart for the week.                                       |

## Scheduled reports

GET /api/report-schedules and PUT /api/report-schedules/:type govern automated emailed reports (weekly team summary, individual CLR summary). The admin sets the cadence (day of week, time of day) and recipients. A manual send-now is also available via POST /api/settings/email/send-now.

## Permissions

Admins and managers can see every CLR. Managers see their direct reports by default and can expand to the full team with a filter. CLRs cannot open this page — they see only their personal version via **My Report** (Chapter 20).

## Related chapters

- **Chapter 20** — My Report
- **Chapter 11** — Stats and Leaderboard
- **Chapter 22** — Team Stats

# My Report

## In this chapter

Your personal version of the team reports page.

## Overview

A personal version of the Reports page. Filter by any date range and see your own outcome breakdown, weekly day-by-day comparison, and goal-vs-actual tracking per metric. Backed by GET /api/my-report.

## What it shows

- Full outcome breakdown (transfers, appointments, callbacks, fell-throughs, future contacts, no-answers).
- Weekly comparison — one row per day with your stats.
- Goal vs actual per metric with color coding (green 100%, yellow 70–99%, red <70%).
- Notes pulled in from your logged outcomes for quick context.
- Personal activity trend chart for the range.

## Date range

The filter defaults to the current week. Choose any custom range. The selection does not persist across sessions — each visit resets to the current week — which is intentional: the page is a daily reflection tool, not a long-term dashboard.

## Goals

Your goals are set by your admin in **Settings Goals**. If nothing is set, the goal column reads em-dash and the progress bars are hidden. Ask your admin to configure targets if they're missing.

## Who can see it

Your own report only. Admins see everyone's data via the team **Reports** page instead (Chapter 19). This page is explicitly private to you.

## Use it to

- Review yesterday's work at the start of the day — which LOs were productive, which weren't.
- Spot patterns: are you burning out on a particular day? Are certain outcome types drifting up?
- Prepare for a 1:1 with your manager. Bring screenshots or CSVs as evidence for asks.

## Related chapters

- **Chapter 19** — Reports
- **Chapter 4** — Dashboard
- **Chapter 11** — Stats and Leaderboard

# Settings (Admin)

## In this chapter

Admin control center: users, goals, assignment generation, algorithm, email, SMS.

## Overview

Settings is the multi-pane surface for user preferences, admin configuration, and org-wide switches. Tabs are gated by role — a CLR sees Profile and Notifications; an admin sees all tabs.

## Tabs

| Tab                  | Audience   | Contents                                           |
|----------------------|------------|----------------------------------------------------|
| Profile              | All        | Name, avatar, timezone, personal contact info.     |
| Notifications        | All        | Toggles for email, push, and in-app notifications. |
| Goals                | Admin      | Per-user daily/weekly/monthly targets.             |
| Scripts              | Admin      | Visual editor for call scripts (Chapter 5).        |
| Integrations         | Admin      | API keys for Mojo, Bonzo, webhook base URL.        |
| Webhook Destinations | Admin      | Outbound webhook configuration (Chapter 30).       |
| Report Schedules     | Admin      | Cadence for automated report emails.               |
| Feature Flags        | SuperAdmin | Beta feature toggles per team or user.             |
| Org                  | SuperAdmin | Org name, SSO domain, retention policy.            |

## Persistence

Settings changes are saved on blur for most fields and on explicit click for toggles. A toast confirms each save. Rollback is possible via the Audit Log (Chapter 24) for admins; users can only see their own history.

## Dangerous settings

A handful of settings — "Delete all my data", "Rotate API keys", "Reset goals" — require a confirmation dialog with a typed confirmation phrase. These actions are always audit-logged with extra context (the requesting user's IP and session id).

## Related chapters

- **Chapter 2** — Roles and Permissions
- **Chapter 6** — Assignments
- **Chapter 23** — Daily Assignment Generation
- **Chapter 26** — Integrations Overview
- **Chapter 44** — Incident Response

# Team Stats

**In this chapter**  
Team-wide metrics framed across the full organization.

## Overview

Team Stats is the manager-facing dashboard for rolled-up team performance. Where LO Stats is per-LO and Reports is weekly, Team Stats lets you compare the whole team day-by-day with in-line sparklines and benchmarks.

## Key panels

| Panel                  | Contents                                                            |
|------------------------|---------------------------------------------------------------------|
| Activity today         | Live count of calls, transfers, appointments since midnight.        |
| 7-day trend            | Sparkline for each metric, day-over-day.                            |
| Team leaderboard       | Top 5 CLRs this week.                                               |
| Bottom watchlist       | Bottom 3 CLRs this week — who might need support.                   |
| Funnel                 | Calls connects transfers appointments, with conversion percentages. |
| Benchmark vs last week | Delta per metric, color-coded.                                      |

## Data sources

- GET /api/team-stats — aggregated snapshot for the selected range.
- GET /api/analytics/history — the trend series for sparklines.
- GET /api/leaderboard — for the top/bottom lists.

## Filtering

Default range is current week. Managers with a scoped team see only their direct reports. Admins see everyone and can filter by team or by individual CLR. Filter state is stored in the URL so dashboards are shareable via link.

## Best practice

Open Team Stats at the start of the day for a live pulse, and again late afternoon as a pacing check. If funnel conversion drops below its trailing average, investigate: which stage slipped (calls, connects, transfers)? Which CLRs drove the delta?

## Related chapters

- **Chapter 11** — Stats and Leaderboard
- **Chapter 19** — Reports

# Daily Assignment Generation

## In this chapter

Generation algorithm, locking, and the unlock-with-reason flow.

## Overview

Daily Assignment is the name of the scheduled job that generates tomorrow's LO assignments for each CLR. It runs every night at 22:00 America/Los\_Angeles and writes to the assignments table.

## Inputs

- Active CLRs: everyone scheduled to work tomorrow (pulled from shift schedule).
- Active LOs: licensed, not on vacation, not marked Unavailable.
- Historical pairings: the last 30 days of CLR-LO assignments, for fairness.
- State coverage matrix: which LOs are licensed in which states.
- LO capacity: a per-LO configurable max number of CLRs per day.

## Algorithm

The engine uses a weighted round-robin. For each CLR, compute a score for each candidate LO based on (1) inverse of recent pairings, (2) state-coverage overlap with the CLR's call mix, (3) LO's remaining capacity for the day. Pick the top N LOs, where N is the configured "LOs per CLR" (default: 4). The algorithm is greedy but balanced — it does not attempt a globally optimal assignment, which would be NP-hard for any real team size.

## Idempotency

If the job is re-run — manually or after a failure — it detects existing assignments for the target date and either skips (default) or overwrites (when run with `--force`). Idempotency avoids duplicate assignment rows.

## Admin override

Admins can pin a specific CLR-LO pairing for a date range in **Settings Assignments**. Pinned pairings are respected over the algorithm's output. Useful for new hires who should shadow a specific senior LO for the first two weeks.

## Monitoring

The job logs a structured summary: number of CLRs assigned, LOs used, average pairings per CLR, and any CLR left under-assigned (which would indicate a coverage gap). The summary is posted to #ops in Chat. A failed job fires an immediate page to on-call.

## Related chapters

- **Chapter 6** — Assignments
- **Chapter 21** — Settings (Admin)
- **Chapter 24** — Audit Log

# Audit Log

## In this chapter

What's audited, what's kept, and how to read the log.

## Overview

The Audit Log records every sensitive action taken in the app. It is admin-only, read-only, and retained for seven years to satisfy compliance requirements.

## Events captured

- Login / logout / session revocation.
- Role changes (promote, demote).
- User creation / deletion.
- Settings changes (goals, scripts, integrations, webhook destinations).
- Impersonation start / stop.
- Data export (CSV download).
- Bulk edits (e.g., mark 12 appointments as Held at once).
- Outcome edits after the day of creation.
- NMLS manual overrides.

- Vacation adds / edits.

## Event shape

| Field       | Detail                                              |
|-------------|-----------------------------------------------------|
| id          | ULID sort key.                                      |
| actor_id    | User who performed the action.                      |
| actor_ip    | Source IP, redacted to /24 for IPv4, /64 for IPv6.  |
| action      | Dotted string, e.g., role.promote.                  |
| target_type | Table name or entity type.                          |
| target_id   | Id of the affected entity.                          |
| before      | JSON snapshot (nullable).                           |
| after       | JSON snapshot (nullable).                           |
| metadata    | Free-form JSON: user agent, session id, request id. |
| occurred_at | UTC timestamp.                                      |

## Search

Filter by actor, target type, action, and date range. Full-text search across metadata via a GIN index. Results stream to a paginated table and can be exported as CSV.

## Retention and tamper-resistance

Audit entries are append-only at the application layer. The underlying table enforces this via a check constraint and a deny-all UPDATE/DELETE policy. Seven-year retention is enforced by a nightly archive job that moves rows older than one year to cold storage and purges rows older than seven years.

## Related chapters

- **Chapter 21** — Settings (Admin)
- **Chapter 23** — Daily Assignment Generation
- **Chapter 25** — Super Admin Panel

# Super Admin Panel

**In this chapter**  
Multi-org management, impersonation, and suspension.

## Overview

Super Admin is the top-of-tree admin surface. It is where an owner promotes or demotes users, adjusts role-level permissions, impersonates a user for debugging, and takes actions that the regular Admin page intentionally does not expose.

## Capabilities

| Capability       | Detail                                                                                                    |
|------------------|-----------------------------------------------------------------------------------------------------------|
| Role management  | Promote user to Admin, demote, or hard-delete. Changes audit-logged.                                      |
| Impersonate      | Open a scoped session as another user for troubleshooting. Banner clearly marks the impersonated session. |
| Invite code      | Generate a one-time-use invite code for a new hire. Configurable expiry.                                  |
| Org settings     | Org-level toggles — SSO domain, webhook base, analytics retention.                                        |
| Feature flags    | Enable or disable beta features per user, per team, or globally.                                          |
| Emergency logout | Immediately revoke all active sessions (e.g., after a credential leak).                                   |

## Impersonation rules

Impersonation is read-only by default — you see the app as the other user but cannot send chat messages, submit outcomes, or perform writes on their behalf. A banner and a distinct border color make the impersonation session visually obvious. The audit log records start and stop timestamps.

## Permissions

Only a user with the SuperAdmin role can open this page. The role is separate from Admin: a SuperAdmin is always an Admin, but an Admin is not automatically a SuperAdmin. By convention the role is held by the org owner and a single backup.

## API endpoints

- POST /api/super/users/:id/promote
- POST /api/super/users/:id/demote
- DELETE /api/super/users/:id
- POST /api/super/impersonate/:id
- POST /api/super/impersonate/stop
- POST /api/super/invites
- POST /api/super/sessions/revoke-all
- GET /api/super/flags / POST /api/super/flags

## Related chapters

- **Chapter 2** — Roles and Permissions
- **Chapter 24** — Audit Log

# Integrations Overview

## In this chapter

Map of all integrations and their admin surface.

## Overview

CLR Connection Center integrates with three external systems: Mojo (dialer, Chapter 27), Bonzo (CRM, Chapter 28), and a generic webhook sink (Chapter 30). This chapter is the big picture — how data flows across systems and where each integration lives in the stack.

## Data flow diagram (textual)

- Dialer side: Mojo places a call CLR answers outcome logged in CLR Connection Center.
- CRM side: Bonzo tracks the borrower record CLR Connection Center writes back outcome and appointment data via Bonzo API calls.
- Webhook side: For each outcome of interest, CLR Connection Center emits a webhook event to the configured URL (Chapter 30) — used for downstream Zapier, n8n, or internal pipelines.

## Auth posture

| Integration | Auth                                                          |
|-------------|---------------------------------------------------------------|
| Mojo        | OAuth token. User connects once; refresh handled server-side. |
| Bonzo       | Long-lived API key per org. Stored in Settings Integrations.  |
| Webhook     | Shared secret used as HMAC signing key on outbound requests.  |

## Failure modes and retries

Each integration has its own retry policy: Mojo uses short exponential backoff (3 tries, 1s/5s/30s), Bonzo uses longer backoff (5 tries over 10 minutes), and webhooks retry hourly for 24 hours. Failed calls are surfaced in **Settings Webhook Events** with the payload and error detail so an admin can replay.

## Observability

GET /api/integrations/health returns a snapshot of last-success and last-failure timestamps per integration. The Monitoring page (Chapter 42) graphs these. Alerts fire when a system has not succeeded in the last 15 minutes during business hours.

## Related chapters

- **Chapter 27** — Mojo Integration
- **Chapter 28** — Bonzo Integration
- **Chapter 29** — Contact Hub
- **Chapter 30** — Webhook Settings

# Mojo Integration

## In this chapter

Mojo dialer ingestion — webhook, sessions, CSV import, public-API stub.

## Overview

Mojo Dialer is the outbound calling platform CLR's use during shifts. CLR Connection Center integrates with Mojo to pull session data, log call metadata, and surface a deep-link back into the dialer for each call.

## Data we consume from Mojo

- Session start/end — when a CLR logs into Mojo and when they sign out.
- Call records — number dialed, duration, disposition.
- Contact records — borrower phone, name if present, pre-populated fields.
- Campaign metadata — which list the contact came from.

## How the integration works

Each org configures a Mojo OAuth connection in **Settings Integrations**. The backend stores an access + refresh token pair. A background poller pulls new call records every 30 seconds during active shifts. When an outcome is logged in CLR Connection Center, the corresponding Mojo record is linked via `mojo_session_id`.

## Sync table

The `mojo_sessions` table is the source of truth for Mojo-linked activity. Rows are keyed by Mojo's internal id and carry the CLR's user id, the contact, the call timestamps, and the disposition. CLR Connection Center outcomes FK into this table.

## Failure modes

- Expired token — the backend attempts a refresh; if that fails, the admin is notified.
- Rate limit — Mojo's API has a burst limit. The poller backs off.
- Stale Mojo records — if Mojo updates a disposition after we pulled it, the next poll picks up the change.
- Missing Mojo record on outcome — allowed; the outcome still saves with `mojo_session_id = null`.

## API surface (internal)

- GET `/api/integrations/mojo/status`
- POST `/api/integrations/mojo/connect`
- POST `/api/integrations/mojo/sync` — manual re-pull.
- DELETE `/api/integrations/mojo/disconnect`

## Related chapters

- **Chapter 26** — Integrations Overview
- **Chapter 29** — Contact Hub
- **Chapter 30** — Webhook Settings

# Bonzo Integration

**In this chapter**

Bonzo CRM prospect sync and push-back.

## Overview

Bonzo is WCL's CRM. CLR Connection Center writes back outcome and appointment data to Bonzo so the downstream loan-officer team has a single source of truth on borrower status. This is a one-way write from our side — we read contact metadata and write activity.

## What we write

| Event              | Bonzo effect                                                              |
|--------------------|---------------------------------------------------------------------------|
| Transfer logged    | Adds a "Transfer received" note to the contact; advances lifecycle stage. |
| Appointment set    | Creates an appointment task in Bonzo's calendar.                          |
| Fell-through       | Adds a tag and a note; does not change stage.                             |
| Callback scheduled | Creates a task due on the callback date.                                  |

## Contact lookup

Before any write, the system looks up the Bonzo contact by phone (normalized E.164). If no contact exists, one is created with the fields we have. If multiple match, the most recently active is used and a warning is logged.

## Resilience

All writes go through a retrying queue with exponential backoff up to 10 minutes total. Persistent failures appear in **Settings Integrations** with a retry button. Writes are idempotent via a `source_event_id` Bonzo stores on our behalf.

## Audit and verification

GET `/api/integrations/bonzo/events?from=&to;=` returns a log of recent writes with their status. Useful for reconciliation at end-of-week.

## Related chapters

- **Chapter 26** — Integrations Overview
- **Chapter 29** — Contact Hub
- **Chapter 30** — Webhook Settings

# Contact Hub

**In this chapter**

Unified contact directory across Mojo, Bonzo, and imports.

## Overview

Contact Hub consolidates the borrower's footprint across systems into one screen — prior Mojo sessions, Bonzo prospect record, recent outcomes, and any scheduled follow-ups. It is the "what do I know about this person right now" page CLR's open mid-call.

## Sections

| Section      | Contents                                                          |
|--------------|-------------------------------------------------------------------|
| Identity     | Name, phone, email, state, NMLS status (from Chapter 18).         |
| Mojo history | List of recent dialer sessions with timestamps and durations.     |
| Bonzo record | Linked CRM record with lifecycle stage, tags, and assigned LO.    |
| Outcomes     | Last 5 outcomes logged in CLR Connection Center for this contact. |
| Appointments | Upcoming and recent appointments involving this contact.          |
| Notes        | Free-form CLR notes, chronological, most recent first.            |

## How it is assembled

Contact Hub is a fan-out call: the server hits Mojo, Bonzo, and the local database in parallel and stitches the results together keyed by phone number and/or email. A soft-deduplication pass merges contacts that match on normalized phone.

## API

- GET /api/contacts/:id/hub — the one-shot aggregate.
- GET /api/contacts/search?q= — fuzzy search for contacts by name, phone, or email.
- POST /api/contacts/:id/notes — add a note.

## Privacy

Notes entered here are visible to other CLRs and admins. Do not log PII beyond what is already in the borrower's own disclosed record (SSN, bank info, etc. must never be typed here). The audit log captures who viewed the hub and when, for compliance review.

## Related chapters

- **Chapter 7** — Directory
- **Chapter 26** — Integrations Overview
- **Chapter 27** — Mojo Integration
- **Chapter 28** — Bonzo Integration

# Webhook Settings

**In this chapter**

Per-integration secrets and the inbound event log.

## Overview

Webhook Settings is the admin page where outbound webhook destinations are configured. Each destination is a URL plus an event filter — "send me every appointment created", "send me only fell-through outcomes", etc. A single app can fan events out to many destinations.

## Destination shape

| Field       | Detail                                                                      |
|-------------|-----------------------------------------------------------------------------|
| URL         | HTTPS endpoint that will receive POST requests.                             |
| Secret      | HMAC-SHA256 signing key. The receiver verifies X-CLR-Signature.             |
| Events      | Multi-select list: appointment.created, outcome.logged, eod.submitted, etc. |
| Active      | Toggle to pause without deleting.                                           |
| Description | Free-text label — "Zapier Slack notifier", "Warehouse sync".                |

## Payload format

Every webhook is a JSON POST with headers: Content-Type: application/json, X-CLR-Event, X-CLR-Timestamp, and X-CLR-Signature. The body is {event, id, occurred\_at, data: {...}} where data is the event payload. Receivers should verify the signature before trusting the body.

## Retries

A non-2xx response triggers retries at 1m, 5m, 30m, 2h, 6h, and 24h. After 24 hours a destination is marked unhealthy and flagged in the admin UI. The event payload is stored in webhook\_events so it can be replayed manually from the UI once the receiver is fixed.

## API

- GET /api/webhook-destinations
- POST /api/webhook-destinations
- PATCH /api/webhook-destinations/:id
- DELETE /api/webhook-destinations/:id
- GET /api/webhook-events?destination=
- POST /api/webhook-events/:id/replay

## Related chapters

- **Chapter 26** — Integrations Overview
- **Chapter 24** — Audit Log

# Frontend Stack

## In this chapter

React 19, Vite, TypeScript, Tailwind, shadcn, Wouter.

## Overview

The frontend is a React 18 single-page application bundled by Vite. It runs entirely in the browser and talks to the backend via the API described in Chapter 34. This chapter walks through the architecture, routing, state management, and build tooling.

## Stack

| Layer      | Technology                                                                               |
|------------|------------------------------------------------------------------------------------------|
| Framework  | React 18 with function components and hooks.                                             |
| Routing    | wouter — a 1 KB client-side router.                                                      |
| State      | TanStack Query (React Query) for server state; React Context for cross-cutting UI state. |
| Forms      | React Hook Form + Zod for validation.                                                    |
| Styling    | Tailwind CSS + shadcn/ui component library.                                              |
| Icons      | lucide-react.                                                                            |
| Build      | Vite 5 with the React plugin; production build outputs to /dist/public.                  |
| TypeScript | Strict mode, noImplicitAny, exactOptionalPropertyTypes.                                  |

## Directory layout

- `client/src/App.tsx` — top-level router and providers.
- `client/src/pages/*.tsx` — one file per route.
- `client/src/components/*` — reusable components, organized by feature.
- `client/src/components/ui/*` — shadcn/ui primitives.
- `client/src/hooks/*` — custom hooks: `useAuth`, `useWebSocket`, `usePushSubscription`, etc.
- `client/src/api/*` — typed fetch wrappers around backend endpoints.
- `client/src/lib/*` — utilities.
- `client/public/*` — static assets served as-is.

## Routing

Client routes are defined in `App.tsx` using `wouter`. Each page is lazy-loaded via `React.lazy` and wrapped in a `Suspense` boundary with a skeleton fallback. Route changes trigger analytics page-views via the analytics hook.

## Data fetching

Every server call goes through TanStack Query. Queries are keyed by the endpoint and parameters; mutations invalidate the relevant keys on success. The default cache time is five minutes; stale-while-revalidate is on. The useAuth query refetches on focus to catch expired sessions.

## Accessibility

All interactive components are built on top of Radix primitives, which provide keyboard navigation and ARIA out of the box. Contrast is tested against WCAG AA. Color is never the sole signal — icons or labels accompany every color-coded state.

## Testing

Component tests use Vitest + React Testing Library. End-to-end smoke tests run Playwright against the staging environment on every deploy. The full suite is in `client/src/__tests__` and `e2e/`.

## Related chapters

- **Chapter 32** — Backend Stack
- **Chapter 33** — Database Schema

# Backend Stack

## In this chapter

Node/Express, better-sqlite3, session auth.

## Overview

The backend is a Node.js application built on Express, written in TypeScript. It terminates HTTPS, authenticates sessions, talks to PostgreSQL, holds WebSocket connections for real-time features, and speaks to external integrations (Mojo, Bonzo, webhook sinks, push service).

## Stack

| Layer           | Technology                                                    |
|-----------------|---------------------------------------------------------------|
| Runtime         | Node.js 20 LTS.                                               |
| Framework       | Express 4 with async route handlers.                          |
| DB              | PostgreSQL 15 via pg and Drizzle ORM.                         |
| Sessions        | express-session with a PG-backed store.                       |
| Auth            | Local auth (argon2 hashing) + OAuth for Google SSO.           |
| WebSockets      | ws library; custom broker on top.                             |
| Push            | web-push library for VAPID push notifications.                |
| Background jobs | lightweight in-process queue — sufficient for current volume. |
| TypeScript      | Strict mode, emits to dist/server/.                           |

## Directory layout

- server/index.ts — startup, graceful shutdown.
- server/routes.ts — Express route table (6000+ lines).
- server/db/ — Drizzle schema and migrations.
- server/services/ — domain services (assignments, reports, integrations).
- server/middleware/ — auth, rate limit, request id, error handler.
- server/ws/ — WebSocket broker and channels.
- shared/ — code shared with the client (types, Zod schemas).

## Request lifecycle

A request lands on Express, receives a request id, is authenticated, rate-limited, and routed to a handler. The handler validates input with Zod, calls one or more services, and returns a JSON response. Errors propagate to the shared error handler which logs and returns a structured error body.

## Database access

All DB access goes through Drizzle. The schema is the single source of truth; migrations are generated from it with drizzle-kit. Connections come from a single pool sized to 10 — tuned for the current load; increase along with the hosting plan.

## Observability

Every request logs a structured line (JSON) with timestamp, request id, user id, route, status, latency, and any error. Logs stream to the platform provider's log collector and are retained for 30 days. Errors also flow to an error-tracking service for alerting.

## Graceful shutdown

On SIGTERM, the server stops accepting new connections, drains in-flight requests with a 30-second deadline, closes WebSocket connections with code 1001, and exits. This matches the hosting platform's rolling-deploy expectations.

## Related chapters

- **Chapter 31** — Frontend Stack
- **Chapter 33** — Database Schema
- **Chapter 35** — Authentication and Sessions

# Database Schema

## In this chapter

The major tables: users, outcomes, eod\_reports, assignments, mojo\_sessions, and more.

## Overview

This chapter documents the database schema that backs CLR Connection Center. Tables are grouped into functional areas: identity and auth, activity capture, integrations, communication, and operational. Every table is owned by the application; there are no shared tables with other systems.

## Conventions

- All primary keys are ULIDs (sortable, unique, url-safe).
- All timestamps are timestampz — UTC.
- Soft-deletes use a deleted\_at column; hard-deletes are rare and documented.
- Foreign keys are always ON DELETE RESTRICT unless noted.
- JSON columns use jsonb with GIN indexes where queried.
- Row-level security is off; auth is enforced at the application layer.

## Identity and auth

| Table              | Purpose                                              |
|--------------------|------------------------------------------------------|
| users              | Every person: CLR, LO, admin, super-admin.           |
| sessions           | Active login sessions keyed by opaque session id.    |
| invites            | One-time codes used for new-hire onboarding.         |
| push_subscriptions | VAPID endpoints for web push.                        |
| orgs               | Multi-tenancy root; most accounts have a single org. |

## Activity capture

| Table                          | Purpose                           |
|--------------------------------|-----------------------------------|
| outcomes                       | Every outcome logged by a CLR.    |
| eod_reports                    | One per CLR per day.              |
| eod_report_activities          | Fine-grained lines inside an EOD. |
| appointments                   | CLR-set borrower appointments.    |
| assignments                    | LO-CLR pairings for a day.        |
| call_scripts+_nodes+_responses | Script graph and telemetry.       |

## Reference data

| Table          | Purpose                                |
|----------------|----------------------------------------|
| loan_officers  | LO master record with NMLS, states.    |
| nmls_checks    | Daily licensing verification results.  |
| glossary_terms | Admin-editable dictionary.             |
| goals          | Per-user daily/weekly/monthly targets. |
| settings       | Org and user preferences.              |

## Integrations

| Table           | Purpose                                  |
|-----------------|------------------------------------------|
| mojo_sessions   | Call records pulled from Mojo.           |
| bonzo_prospects | Cached Bonzo contact records.            |
| webhook_events  | Outbound event log for retry and replay. |

## Communication

| Table                     | Purpose                                       |
|---------------------------|-----------------------------------------------|
| chat_messages             | Every chat post, including edits and deletes. |
| forum_posts+forum_answers | Q&A; threads and replies.                     |
| notifications             | Per-user inbox of in-app notifications.       |

## Operational

| Table     | Purpose                                                |
|-----------|--------------------------------------------------------|
| audit_log | Append-only history of sensitive actions (Chapter 24). |

## Migrations

Migrations are managed by Drizzle Kit. Each migration is a forward-only SQL file checked into server/db/migrations/. Rollbacks are performed via a new migration, not by running a down migration. The deploy pipeline runs drizzle-kit migrate before starting the server; a migration failure fails the deploy.

## Related chapters

- **Chapter 32** — Backend Stack
- **Chapter 34** — API Reference

# API Reference

## In this chapter

Grouped endpoint reference for all 184 routes.

## How to read this reference

The API reference is organized by URL prefix. Every endpoint is grouped by its `/api/<prefix>`. Within a group, endpoints are ordered by method then path. A brief description is given for each group; notes are provided for individual endpoints where behavior is non-obvious.

## Conventions

- All endpoints are prefixed with `/api`.
- All timestamps are ISO 8601 UTC with a Z suffix.
- Authentication: session cookie for browser clients; bearer token for integrations.
- Content type: `application/json` for both request and response bodies.
- Errors: non-2xx responses return `{error: {code, message, details}}`.
- Pagination: `?limit=N&cursor;=OPAQUE` where supported; response includes `nextCursor`.

- Rate limits: per-user 120 req/min soft, 600 req/min hard. 429 on breach.

## Authentication errors

| Code                | Meaning                                                    |
|---------------------|------------------------------------------------------------|
| 401 unauthenticated | No valid session cookie or bearer token. Log in again.     |
| 403 forbidden       | Authenticated but role lacks permission for this endpoint. |
| 419 session_expired | Session TTL elapsed. Client should refresh.                |

## Endpoint groups

Each group below corresponds to a `/api/<prefix>` namespace. For each group, this section describes what the group does, who can call it, any shared behaviors, and lists every endpoint with method, path, and per-endpoint notes where the behavior is non-obvious.

### /api/admin

Administrative utilities (hotpatch).

**Access:** authenticated users by default; admin-only where marked in the note column.

| Method | Path                             | Note                                               |
|--------|----------------------------------|----------------------------------------------------|
| POST   | <code>/api/admin/hotpatch</code> | Emergency hotpatch endpoint (audited, admin only). |

### /api/analytics

Historical analytics series.

**Access:** authenticated users by default; admin-only where marked in the note column.

| Method | Path                                | Note                                            |
|--------|-------------------------------------|-------------------------------------------------|
| GET    | <code>/api/analytics/history</code> | Activity trend history for 1D/1W/1M/All charts. |

### /api/assignment-overrides

Per-day admin overrides applied on top of auto-generated assignments.

**Access:** authenticated users by default; admin-only where marked in the note column.

| Method | Path                      | Note |
|--------|---------------------------|------|
| GET    | /api/assignment-overrides |      |

## /api/assignments

Daily assignment generation, overrides, and per-day records.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                                 | Note                                                          |
|--------|--------------------------------------|---------------------------------------------------------------|
| GET    | /api/assignments                     |                                                               |
| PATCH  | /api/assignments/:id                 |                                                               |
| POST   | /api/assignments/:id/admin-override  | Admin: per-row override on an assignment.                     |
| POST   | /api/assignments/generate            | Admin action: generates one assignment per CLR for today.     |
| POST   | /api/assignments/regenerate-override | Admin: unlocks and regenerates today with a reason (audited). |
| GET    | /api/assignments/today               |                                                               |

**Example:** `curl -H 'Cookie: session=...' https://app.wlc.it.com/api/assignments` returns the list as JSON. Pagination where supported uses `?limit=&cursor=`.

## /api/audit-logs

Audit trail of administrative actions.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path            | Note                   |
|--------|-----------------|------------------------|
| GET    | /api/audit-logs | Paginated audit trail. |

**Example:** `curl -H 'Cookie: session=...' https://app.wlc.it.com/api/audit-logs` returns the list as JSON. Pagination where supported uses `?limit=&cursor=`.

## /api/auth

Authentication, session management, and password flows.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                      | Note                                                             |
|--------|---------------------------|------------------------------------------------------------------|
| POST   | /api/auth/change-password | Authenticated password change (current password required).       |
| POST   | /api/auth/forgot-password | Issues a reset token and emails a reset link.                    |
| POST   | /api/auth/login           | Authenticates with email and password; creates a session cookie. |
| POST   | /api/auth/logout          | Destroys the current session.                                    |
| GET    | /api/auth/me              | Returns the currently logged-in user and role flags.             |
| PATCH  | /api/auth/profile         | Updates the current user's profile fields.                       |
| POST   | /api/auth/reset-password  | Completes the password reset with the emailed token.             |

## /api/bonzo

Bonzo CRM prospects, pipelines, sync, and push-back operations.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                      | Note                                                     |
|--------|---------------------------|----------------------------------------------------------|
| GET    | /api/bonzo/pipelines      |                                                          |
| GET    | /api/bonzo/prospects      |                                                          |
| POST   | /api/bonzo/push/note      |                                                          |
| POST   | /api/bonzo/push/outcome   | Push a CLR outcome back to Bonzo as a note / stage move. |
| POST   | /api/bonzo/push/stage     |                                                          |
| GET    | /api/bonzo/sync-log       |                                                          |
| POST   | /api/bonzo/sync/full      | Full Bonzo prospect + pipeline sync.                     |
| POST   | /api/bonzo/sync/pipelines |                                                          |
| POST   | /api/bonzo/sync/prospects |                                                          |

## /api/call-logs

Raw call log rows (Mojo-sourced or direct).

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                              | Note |
|--------|-----------------------------------|------|
| GET    | /api/call-logs                    |      |
| POST   | /api/call-logs                    |      |
| GET    | /api/call-logs/check-previous-day |      |
| GET    | /api/call-logs/summary            |      |

## /api/call-scripts

Versioned calling scripts with branching nodes and responses.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                               | Note                                     |
|--------|------------------------------------|------------------------------------------|
| GET    | /api/call-scripts                  | List scripts available to this user.     |
| POST   | /api/call-scripts                  | Create a new script (admin).             |
| DELETE | /api/call-scripts/:id              |                                          |
| PATCH  | /api/call-scripts/:id              |                                          |
| POST   | /api/call-scripts/:id/clone        | Clone a script into a new editable copy. |
| GET    | /api/call-scripts/:id/node/:nodeId | Single script node.                      |
| POST   | /api/call-scripts/:id/nodes        | Add a node to a script.                  |
| GET    | /api/call-scripts/:id/root         | Root node of a script.                   |
| GET    | /api/call-scripts/:id/tree         | Full branching tree for a script.        |
| GET    | /api/call-scripts/defaults         | The default script template.             |
| DELETE | /api/call-scripts/mine             | Revert to the default script.            |
| GET    | /api/call-scripts/mine             | Get this user's active script.           |

## /api/chat

Team chat messages (global room).

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path      | Note |
|--------|-----------|------|
| GET    | /api/chat |      |

| Method | Path          | Note                                       |
|--------|---------------|--------------------------------------------|
| POST   | /api/chat     | Posts a new chat message to the team room. |
| DELETE | /api/chat/:id | Deletes a chat message (author or admin).  |

**Example:** `curl -H 'Cookie: session=...' https://app.wlc.it.com/api/chat` returns the list as JSON. Pagination where supported uses `?limit=&cursor=`.

## /api/contacts

Unified Contact Hub across Mojo, Bonzo, and direct imports.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path              | Note |
|--------|-------------------|------|
| GET    | /api/contacts     |      |
| GET    | /api/contacts/:id |      |

## /api/dashboard

Dashboard widgets — stats and recent activity.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path                           | Note                                    |
|--------|--------------------------------|-----------------------------------------|
| GET    | /api/dashboard/recent-activity | Recent activity list for the Dashboard. |
| GET    | /api/dashboard/stats           | KPI tiles for the Dashboard.            |

## /api/eod

Lightweight draft endpoints for the EOD report form.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path           | Note                                                   |
|--------|----------------|--------------------------------------------------------|
| DELETE | /api/eod/draft |                                                        |
| GET    | /api/eod/draft |                                                        |
| PUT    | /api/eod/draft | Upserts the draft EOD report (auto-save while typing). |

## /api/eod-reports

End-of-Day report submissions, drafts, history, and activity log.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                            | Note                                                   |
|--------|---------------------------------|--------------------------------------------------------|
| GET    | /api/eod-reports                |                                                        |
| POST   | /api/eod-reports                | Submits today's EOD report (locks from further edits). |
| POST   | /api/eod-reports/activities     |                                                        |
| DELETE | /api/eod-reports/activities/:id |                                                        |
| GET    | /api/eod-reports/history        | Lists past submitted EOD reports (admin can see team). |

**Example:** `curl -H 'Cookie: session=...' https://app.wlc.it.com/api/eod-reports` returns the list as JSON. Pagination where supported uses `?limit=&cursor=`.

## /api/export

CSV exports for outcomes, LOs, users, and daily logs.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                      | Note                           |
|--------|---------------------------|--------------------------------|
| GET    | /api/export/daily-logs    | CSV export of daily call logs. |
| GET    | /api/export/loan-officers | CSV export of LO roster.       |
| GET    | /api/export/outcomes      | CSV export of outcomes.        |
| GET    | /api/export/users         | CSV export of users.           |

## /api/forum

Forum posts, answers, upvotes, subscriptions, and accepted answers.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                   | Note |
|--------|------------------------|------|
| DELETE | /api/forum/answers/:id |      |
| PATCH  | /api/forum/answers/:id |      |

| Method | Path                                             | Note                                            |
|--------|--------------------------------------------------|-------------------------------------------------|
| POST   | /api/forum/answers/:id/upvote                    | Upvote an answer on a post.                     |
| GET    | /api/forum/posts                                 |                                                 |
| POST   | /api/forum/posts                                 | Creates a new forum post.                       |
| DELETE | /api/forum/posts/:id                             |                                                 |
| GET    | /api/forum/posts/:id                             |                                                 |
| PATCH  | /api/forum/posts/:id                             |                                                 |
| POST   | /api/forum/posts/:id/accept-answer/:answer<br>Id | Mark an answer as the accepted one.             |
| POST   | /api/forum/posts/:id/answers                     |                                                 |
| POST   | /api/forum/posts/:id/subscribe                   | Subscribe to a post's new-answer notifications. |
| POST   | /api/forum/posts/:id/upvote                      | Upvote a forum post.                            |

**Example:** curl -H 'Cookie: session=...' https://app.wlc.it.com/api/forum returns the list as JSON. Pagination where supported uses ?limit=&cursor=.

## /api/glossary

CLR/industry glossary terms (admin-managed).

**Access:** authenticated users by default; admin-only where marked in the note column.

| Method | Path              | Note                         |
|--------|-------------------|------------------------------|
| GET    | /api/glossary     |                              |
| POST   | /api/glossary     | Add a glossary term (admin). |
| DELETE | /api/glossary/:id | Delete a glossary term.      |
| PATCH  | /api/glossary/:id | Edit a glossary term.        |

**Example:** curl -H 'Cookie: session=...' https://app.wlc.it.com/api/glossary returns the list as JSON. Pagination where supported uses ?limit=&cursor=.

## /api/goals

Per-user performance goals.

**Access:** authenticated users by default; admin-only where marked in the note column.

| Method | Path               | Note |
|--------|--------------------|------|
| GET    | /api/goals         |      |
| GET    | /api/goals/:userId |      |
| PATCH  | /api/goals/:userId |      |

## /api/health

Health check for uptime monitors.

**Access:** authenticated users by default; admin-only where marked in the note column.

| Method | Path        | Note                                                       |
|--------|-------------|------------------------------------------------------------|
| GET    | /api/health | Returns {status, uptime, db} — used by Railway / monitors. |

## /api/integrations

Integration status summary.

**Access:** authenticated users by default; admin-only where marked in the note column.

| Method | Path                     | Note                                                      |
|--------|--------------------------|-----------------------------------------------------------|
| GET    | /api/integrations/status | Summary of Mojo / Bonzo / SMS / Email integration health. |

## /api/invite

Invite token resolution and acceptance.

**Access:** authenticated users by default; admin-only where marked in the note column.

| Method | Path                      | Note |
|--------|---------------------------|------|
| GET    | /api/invite/:token        |      |
| POST   | /api/invite/:token/accept |      |

## /api/leaderboard

Team leaderboard data source.

**Access:** authenticated users by default; admin-only where marked in the note column.

| Method | Path             | Note                                            |
|--------|------------------|-------------------------------------------------|
| GET    | /api/leaderboard | Ranked team stats used by the Leaderboard page. |

**Example:** `curl -H 'Cookie: session=...' https://app.wlc.it.com/api/leaderboard` returns the list as JSON. Pagination where supported uses `?limit=&cursor=`.

## /api/loan-officers

LO directory, credentials, snoozing, performance.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path                                | Note                                        |
|--------|-------------------------------------|---------------------------------------------|
| GET    | /api/loan-officers                  |                                             |
| POST   | /api/loan-officers                  | Create a loan officer.                      |
| DELETE | /api/loan-officers/:id              | Archive a loan officer.                     |
| GET    | /api/loan-officers/:id              |                                             |
| PATCH  | /api/loan-officers/:id              | Update LO fields.                           |
| GET    | /api/loan-officers/:id/availability |                                             |
| PUT    | /api/loan-officers/:id/availability | Mark LO available / unavailable (vacation). |
| GET    | /api/loan-officers/:id/credentials  |                                             |
| GET    | /api/loan-officers/:id/performance  | Per-LO transfer / call performance.         |
| PATCH  | /api/loan-officers/:id/status       |                                             |
| POST   | /api/loan-officers/import           | Bulk-import LOs from CSV.                   |
| GET    | /api/loan-officers/snoozed          | List of LOs currently marked unavailable.   |

## /api/me

Current-user convenience endpoints.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path    | Note |
|--------|---------|------|
| GET    | /api/me |      |

## /api/mojo

Mojo dialer sessions, contacts, sync log, and CSV import.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path                    | Note                                           |
|--------|-------------------------|------------------------------------------------|
| GET    | /api/mojo/contacts      |                                                |
| POST   | /api/mojo/import/csv    | Bulk-import historical Mojo sessions from CSV. |
| GET    | /api/mojo/sessions      |                                                |
| GET    | /api/mojo/sync-log      |                                                |
| POST   | /api/mojo/sync/sessions |                                                |

## /api/monthly-assignments

Monthly rotation management and shuffle operations.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path                             | Note                             |
|--------|----------------------------------|----------------------------------|
| GET    | /api/monthly-assignments         |                                  |
| POST   | /api/monthly-assignments/shuffle | Shuffle the monthly LO rotation. |

## /api/my-goals

Personal goals update.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path          | Note |
|--------|---------------|------|
| PUT    | /api/my-goals |      |

## /api/my-report

Personal admin report endpoint.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path           | Note |
|--------|----------------|------|
| GET    | /api/my-report |      |

**Example:** curl -H 'Cookie: session=...' https://app.wlc.it.com/api/my-report returns the list as JSON. Pagination where supported uses ?limit=&cursor=.

## /api/nmls

NMLS license validity checks for loan officers.

**Access:** authenticated users by default; admin-only where marked in the note column.

| Method | Path                          | Note                                     |
|--------|-------------------------------|------------------------------------------|
| POST   | /api/nmls/check-all           | Re-runs NMLS license checks for all LOs. |
| POST   | /api/nmls/check/:loId         | Runs NMLS license check for a single LO. |
| POST   | /api/nmls/mark-verified/:loId |                                          |
| GET    | /api/nmls/status              |                                          |

**Example:** curl -H 'Cookie: session=...' https://app.wlc.it.com/api/nmls returns the list as JSON. Pagination where supported uses ?limit=&cursor=.

## /api/nmls-checks

Queue of pending NMLS checks and triggers.

**Access:** authenticated users by default; admin-only where marked in the note column.

| Method | Path                           | Note |
|--------|--------------------------------|------|
| GET    | /api/nmls-checks               |      |
| POST   | /api/nmls-checks/:loId/confirm |      |
| GET    | /api/nmls-checks/my-pending    |      |
| POST   | /api/nmls-checks/trigger       |      |

## /api/nmls-schedule

NMLS automatic-check schedule (cadence and toggles).

**Access:** authenticated users by default; admin-only where marked in the note column.

| Method | Path               | Note |
|--------|--------------------|------|
| GET    | /api/nmls-schedule |      |
| PATCH  | /api/nmls-schedule |      |

## /api/notifications

In-app notification list, unread count, mark-read.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path                             | Note                                    |
|--------|----------------------------------|-----------------------------------------|
| GET    | /api/notifications               | List in-app notifications.              |
| POST   | /api/notifications               |                                         |
| PATCH  | /api/notifications/:id/read      | Mark a notification read.               |
| POST   | /api/notifications/mark-all-read | Mark all notifications read.            |
| GET    | /api/notifications/unread-count  | Unread notification count for the bell. |

## /api/org

Current organization metadata.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path             | Note |
|--------|------------------|------|
| GET    | /api/org/current |      |

## /api/orgs

Organization-level invite management.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path                            | Note |
|--------|---------------------------------|------|
| POST   | /api/orgs/:id/invite            |      |
| GET    | /api/orgs/:id/invites           |      |
| DELETE | /api/orgs/:id/invites/:inviteId |      |

## /api/outcomes

Call outcome records — the heart of CLR activity tracking.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path              | Note                                                               |
|--------|-------------------|--------------------------------------------------------------------|
| GET    | /api/outcomes     |                                                                    |
| POST   | /api/outcomes     | Creates a call outcome (transfer, appointment, fell through, ...). |
| DELETE | /api/outcomes/:id | Soft-deletes the outcome (admin only in most cases).               |
| PATCH  | /api/outcomes/:id | Updates an outcome — e.g., changes status after the fact.          |

**Example:** `curl -H 'Cookie: session=...' https://app.wlc.it.com/api/outcomes` returns the list as JSON. Pagination where supported uses `?limit=&cursor=`.

## /api/push

Web Push subscriptions, VAPID key, and broadcast.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path                       | Note                                                       |
|--------|----------------------------|------------------------------------------------------------|
| POST   | /api/push/send             |                                                            |
| POST   | /api/push/subscribe        | Subscribe browser to Web Push (adds PushSubscription row). |
| DELETE | /api/push/unsubscribe      | Unsubscribe Web Push for this device.                      |
| GET    | /api/push/vapid-public-key | Returns the VAPID public key used by the PWA to subscribe. |

## /api/report-schedules

Scheduled automated reports.

*Access:* authenticated users by default; admin-only where marked in the note column.

| Method | Path                        | Note |
|--------|-----------------------------|------|
| GET    | /api/report-schedules       |      |
| PUT    | /api/report-schedules/:type |      |

## /api/reporting-period

Reporting period definitions (weeks).

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                  | Note |
|--------|-----------------------|------|
| GET    | /api/reporting-period |      |

## /api/request-access

Public self-serve access request.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                | Note                                    |
|--------|---------------------|-----------------------------------------|
| POST   | /api/request-access | Public form — asks admin for an invite. |

## /api/script-nodes

Individual branch nodes of a calling script.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                                | Note                      |
|--------|-------------------------------------|---------------------------|
| DELETE | /api/script-nodes/:id               | Delete a script node.     |
| PATCH  | /api/script-nodes/:id               | Update a script node.     |
| POST   | /api/script-nodes/:nodeId/responses | Add a response to a node. |

## /api/script-responses

Responses attached to a script node.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                      | Note               |
|--------|---------------------------|--------------------|
| DELETE | /api/script-responses/:id | Delete a response. |
| PATCH  | /api/script-responses/:id | Update a response. |

## /api/settings

Admin configuration — algorithm weights, email settings.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                         | Note                                   |
|--------|------------------------------|----------------------------------------|
| GET    | /api/settings/algorithm      | Assignment algorithm weights.          |
| PATCH  | /api/settings/algorithm      | Update algorithm weights.              |
| GET    | /api/settings/email          | Email settings (SMTP, from, reply-to). |
| PATCH  | /api/settings/email          | Update email settings.                 |
| POST   | /api/settings/email/send-now | Trigger queued emails immediately.     |
| POST   | /api/settings/email/test     | Send a diagnostic email.               |

## /api/sms

SMS integration settings, status, and test sends.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path              | Note                   |
|--------|-------------------|------------------------|
| GET    | /api/sms/settings | Stored SMS settings.   |
| PATCH  | /api/sms/settings | Update SMS settings.   |
| GET    | /api/sms/status   | SMS gateway status.    |
| POST   | /api/sms/test     | Send a diagnostic SMS. |

## /api/stats

Aggregate stats endpoint used by multiple pages.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path       | Note |
|--------|------------|------|
| GET    | /api/stats |      |

## /api/status

Lightweight public status endpoint.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path        | Note                                |
|--------|-------------|-------------------------------------|
| GET    | /api/status | Public, cache-friendly status ping. |

## /api/super-admin

Multi-org management for super admins.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                                  | Note                                   |
|--------|---------------------------------------|----------------------------------------|
| POST   | /api/super-admin/exit-impersonate     | End super-admin impersonation session. |
| GET    | /api/super-admin/orgs                 |                                        |
| POST   | /api/super-admin/orgs                 |                                        |
| PATCH  | /api/super-admin/orgs/:id             |                                        |
| POST   | /api/super-admin/orgs/:id/impersonate | Super admin: impersonate another org.  |
| DELETE | /api/super-admin/orgs/:id/suspend     | Super admin: suspend an organization.  |
| POST   | /api/super-admin/stop-impersonating   |                                        |

## /api/test-email

Diagnostic email send.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path            | Note                        |
|--------|-----------------|-----------------------------|
| POST   | /api/test-email | Send a test email (legacy). |

## /api/user

Per-user getting-started state.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                      | Note                                        |
|--------|---------------------------|---------------------------------------------|
| GET    | /api/user/getting-started | Get the current user's onboarding progress. |
| POST   | /api/user/getting-started | Record progress on an onboarding step.      |

## /api/users

User CRUD, role assignment, welcome emails, and profile.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                          | Note                                         |
|--------|-------------------------------|----------------------------------------------|
| GET    | /api/users                    |                                              |
| POST   | /api/users                    | Create a user (admin).                       |
| DELETE | /api/users/:id                | Archive a user (admin).                      |
| PATCH  | /api/users/:id                | Edit a user (admin).                         |
| PATCH  | /api/users/:id/manager        | Assign this user's manager.                  |
| POST   | /api/users/:id/resend-welcome | Resend the welcome / invite email.           |
| PATCH  | /api/users/:id/reset-intro    | Clear the 'getting started' flag for a user. |
| PUT    | /api/users/me/password        | Change password (self-serve).                |
| PATCH  | /api/users/me/seen-intro      | Mark the intro video as watched.             |
| PATCH  | /api/users/me/sms-reminders   | Toggle SMS reminders on/off.                 |

## /api/webhook

Inbound webhooks from Mojo/Bonzo/Zapier and settings.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                     | Note                                              |
|--------|--------------------------|---------------------------------------------------|
| POST   | /api/webhook/bonzo       | Inbound webhook: Bonzo prospect events.           |
| GET    | /api/webhook/events      |                                                   |
| POST   | /api/webhook/mojo        | Inbound webhook: Mojo session ingest.             |
| POST   | /api/webhook/mojo/zapier | Inbound webhook: Mojo via Zapier passthrough.     |
| GET    | /api/webhook/settings    |                                                   |
| PUT    | /api/webhook/settings    | Updates stored secrets and URLs for integrations. |

## /api/zapier

Outbound Zapier triggers.

Access: authenticated users by default; admin-only where marked in the note column.

| Method | Path                | Note                     |
|--------|---------------------|--------------------------|
| POST   | /api/zapier/trigger | Outbound Zapier webhook. |

## Client libraries

An auto-generated OpenAPI spec is available at `/api/_openapi.json`. Feed it to your code-gen tool of choice — `openapi-generator`, `orval`, `oapi-codegen`. A typed TypeScript SDK is checked into the repo under `client/src/api/` and is the canonical client for the web app.

## Versioning and deprecation

The API is not SemVer-versioned. Breaking changes are communicated in the changelog (Chapter 45) and via the response header `X-CLR-Deprecation`, which carries the sunset date for any endpoint scheduled for removal. Minimum 60-day deprecation window.

## Related chapters

- **Chapter 32** — Backend Stack
- **Chapter 33** — Database Schema
- **Chapter 35** — Authentication and Sessions

# Authentication and Sessions

## In this chapter

Cookie sessions, password flows, invites, and role guards.

## Overview

Authentication is session-based for browser clients and bearer-token-based for integrations. Sessions are stored server-side in PostgreSQL. Tokens are opaque. Passwords are hashed with argon2id. This chapter walks through the full auth lifecycle.

## Login flow

- 1. User submits email + password to POST /api/auth/login.
- 2. Server looks up the user by email, verifies the password with argon2.
- 3. On success, server creates a session row with a random 256-bit id, sets an httpOnly, secure, SameSite=Lax cookie.
- 4. Subsequent requests carry the cookie; a middleware loads the session on every request.
- 5. Failed logins increment a per-email counter. 10 failures in 15 minutes locks the account for 30 minutes.

## SSO

Google SSO is supported via OAuth 2.0. Users whose email domain matches the org's configured SSO domain (**Settings Org**) can click "Sign in with Google" instead of typing a password. A successful Google sign-in creates a session the same way as a password login.

## Password policy

- Minimum 12 characters.
- Checked against the HaveIBeenPwned password list.
- Cannot match the last 5 passwords used.
- Rotated on admin request via **Super Admin User Reset Password**.

## Session lifetime

A session TTL is 30 days of inactivity, with a hard cap of 90 days total. The server refreshes the last\_seen\_at timestamp on every authenticated request. Sessions can be individually revoked ("sign out" button) or bulk-revoked ("sign out everywhere").

## Bearer tokens

Integrations use long-lived bearer tokens generated by a SuperAdmin in **Super Admin API Tokens**. Each token has a scope (read-only or full) and an optional expiration. Tokens are displayed once at creation and hashed at rest; losing a token means rotating it.

## CSRF

Since sessions are cookie-based, CSRF is relevant. The backend uses a double-submit cookie pattern: the client reads csrf\_token and includes it as X-CSRF-Token on write requests. Same-site Lax cookies and Origin checks are layered defenses.

## Related chapters

- **Chapter 32** — Backend Stack
- **Chapter 34** — API Reference

# Progressive Web App

## In this chapter

Manifest, service worker wclcc-v2, icon set, install flow.

## Overview

CLR Connection Center is a Progressive Web App. Users can install it on desktop and mobile, receive push notifications, and use a limited subset of features offline. This chapter documents the PWA surface.

## Install

- Desktop (Chrome/Edge): a + icon appears in the address bar when the user has visited the app twice.
- iOS Safari: Share Add to Home Screen. Installs as a standalone app icon.
- Android Chrome: browser prompts with "Add to Home Screen" banner after engagement.
- Once installed, the app opens in its own window with no browser chrome.

## Service worker

The service worker is built with Workbox. It implements: (1) offline fallback page for navigation requests, (2) cache-first strategy for static assets (JS, CSS, fonts), (3) network-first for API calls with a stale-while-revalidate fallback, (4) push event handler for web-push notifications.

## Cache strategy

| Asset type   | Strategy                      | TTL       |
|--------------|-------------------------------|-----------|
| JS/CSS/fonts | Cache-first, hashed filenames | Immutable |
| Images       | Cache-first                   | 30 days   |
| API GETs     | Network-first with fallback   | 5 minutes |
| HTML shell   | Network-first                 | 1 hour    |

## Offline capabilities

When offline, the user sees an offline banner. They can read previously-loaded pages (from cache) and draft outcomes locally; drafts sync when the connection returns. Mutation queueing uses IndexedDB. The user's team feed and chat freeze until reconnected.

## Cache-bust strategy

A build produces hashed filenames; the shell's index.html is revalidated on every load via network-first. New service workers activate on next navigation after a 12-second grace period. See Chapter 41 for the full cache-bust chapter.

## Related chapters

- **Chapter 3** — Getting Started
- **Chapter 39** — Web Push Notifications
- **Chapter 41** — Cache Busting and Redeployment

# Timezone Handling

**In this chapter**

The `parseDbTimestamp` helper and UTC-to-local normalization.

## Overview

CLR Connection Center is used across U.S. time zones. The app stores every timestamp in UTC and renders in the viewer's local time zone. This chapter documents the conventions.

## Storage vs display

| Layer     | Convention                                                                          |
|-----------|-------------------------------------------------------------------------------------|
| Database  | UTC. All timestamp columns are <code>timestamptz</code> .                           |
| API       | ISO 8601 with explicit Z suffix. Never naive.                                       |
| Client    | Converts to the user's OS timezone for display. Shows IANA name on hover.           |
| Reports   | Weekly boundaries computed in <code>America/Los_Angeles</code> — the business week. |
| Cron jobs | Scheduled in <code>America/Los_Angeles</code> to align with business day.           |

## Reporting weeks

A reporting week runs Monday 00:00 through Sunday 23:59:59 in America/Los\_Angeles. This is the WCL business week. The boundary is intentionally not per-user because comparing CLRs requires a common denominator.

## User preference

Users can override the display timezone in Settings Profile. The default is the browser-reported timezone. Changing it affects only rendering, never storage.

## Edge cases

- DST transitions: the weekly boundary may fall at a missing or duplicated local time. The app uses Luxon and treats the UTC instant as authoritative.
- Midnight-straddling calls: a call starting 23:55 local and ending 00:05 local counts for the day it started.
- Cross-timezone appointments: the appointment's time is stored in UTC and displayed in each viewer's timezone; both are shown in the confirmation email to avoid ambiguity.

## Related chapters

- **Chapter 32** — Backend Stack
- **Chapter 33** — Database Schema

# Realtime Features

## In this chapter

Polling intervals, visibility handling, and where it's used.

## Overview

Real-time features in CLR Connection Center ride a single WebSocket connection per client. The backend broker multiplexes multiple logical channels over that one connection: chat, activity feed, notifications, and assignment updates.

## Handshake

The client opens a WebSocket to /ws with the session cookie attached. The server reads the cookie, verifies the session, and upgrades the connection. An immediate `{type:'hello', userId, channels}` message confirms the server's view.

## Channel model

| Channel                | Purpose                                                     |
|------------------------|-------------------------------------------------------------|
| chat:<channelId>       | New messages, edits, deletes, reactions, typing indicators. |
| activity:team          | Every outcome logged by any teammate (dashboard feed).      |
| notifications:<userId> | Personal notifications: mentions, assignment changes.       |
| assignments:<userId>   | Live assignment updates when the engine rebalances.         |

## Reconnection

The client auto-reconnects on drop with exponential backoff starting at 1 s, capped at 30 s. On reconnect, it sends {type:'resume', lastEventId}, and the server replays missed events where possible. Events older than 5 minutes are discarded; the client refetches via HTTP to catch up.

## Heartbeats

The server sends a ping every 30 seconds; the client must reply within 10 seconds or the connection is closed with code 1011. This catches half-open connections behind proxies.

## Scaling

The current deployment is single-process, which supports a few hundred concurrent connections comfortably. When it is time to horizontally scale, the broker will need a pub/sub backend (Redis or NATS) so events cross process boundaries. The broker's internal API is designed with that swap in mind.

## Related chapters

- **Chapter 12** — Chat
- **Chapter 10** — Appointments
- **Chapter 4** — Dashboard

# Web Push Notifications

## In this chapter

VAPID keys, the push-subscribe flow, and when push fires.

## Overview

Push notifications let the app reach CLRs when the tab isn't open. Notifications are delivered via the standard Web Push protocol using VAPID. They are opt-in, configurable per category, and can be disabled at any time.

## Categories

- Chat mentions — when someone @mentions you.
- Direct messages — any DM.
- Assignment changes — mid-day reassignments affecting you.
- Appointment reminders — 24h and 1h before a scheduled appointment.
- NMLS alerts — licensing expirations within 30 days.
- System alerts — outages, admin broadcasts.

## Subscription flow

- 1. User clicks "Enable notifications" in Settings.
- 2. Browser prompts for permission.
- 3. On permission grant, the service worker generates a push subscription.
- 4. Client sends the subscription to POST /api/push/subscribe.
- 5. Server stores it in push\_subscriptions keyed by user + endpoint.

## Delivery

When an event fires that warrants a push, the backend builds a notification payload (title, body, url, icon) and sends it to the stored endpoint with web-push. The service worker receives it, displays the notification, and a click takes the user to the target URL. Delivery typically takes under 2 seconds.

## VAPID keys

VAPID keys are generated once per deployment and stored in the secret manager as VAPID\_PUBLIC\_KEY and VAPID\_PRIVATE\_KEY. Rotating them invalidates all active push subscriptions, so clients must re-subscribe. Rotation is a once-a-year operation unless a key leaks.

## Reliability

Push is best-effort. Browsers drop notifications when the device is offline, on low battery, or the endpoint is stale. A 410 response from the push service indicates a dead subscription and the backend deletes it. Critical information must never rely on push alone — always also surface in-app.

## Related chapters

- **Chapter 3** — Getting Started
- **Chapter 36** — Progressive Web App
- **Chapter 38** — Realtime Features

# Deployment to Railway

**In this chapter**

Railway deployment, Dockerfile, branch binding, auto-deploy.

## Overview

CLR Connection Center is deployed on Railway — a container-based platform. The app runs as a single Dockerized process fronted by Railway's HTTPS edge. This chapter covers the deploy pipeline, environment variables, and scaling knobs.

## Project layout on Railway

| Resource          | Detail                                                               |
|-------------------|----------------------------------------------------------------------|
| App service       | Node.js runtime, auto-scaled to one instance initially.              |
| PostgreSQL add-on | Managed PG 15, 5 GB storage tier to start.                           |
| Object storage    | Used for uploads and backups. Railway-provided S3-compatible bucket. |

## Deploy pipeline

- 1. Push to main triggers a Railway build.
- 2. npm ci installs dependencies.

- 3. npm run build bundles frontend (Vite) and compiles backend (tsc).
- 4. drizzle-kit migrate applies pending migrations.
- 5. Container starts, hits /api/ready; Railway swaps traffic after a 200.
- 6. Old container is drained and killed.

## Environment variables

| Variable              | Purpose                                   |
|-----------------------|-------------------------------------------|
| DATABASE_URL          | Postgres connection string.               |
| SESSION_SECRET        | Cookie signing key.                       |
| VAPID_PUBLIC_KEY      | Web push public key.                      |
| VAPID_PRIVATE_KEY     | Web push private key.                     |
| MOJO_CLIENT_ID/SECRET | Mojo OAuth credentials.                   |
| BONZO_API_KEY         | Bonzo CRM key.                            |
| NODE_ENV              | Must be production.                       |
| PORT                  | Injected by Railway; the app binds to it. |

## Rollback

Railway keeps the last 10 deploys. Rollback is one click: navigate to Deployments, select the last known-good version, click Restore. Database migrations are not auto-rolled back — if a migration is incompatible with the previous code, a forward-only migration fixing the issue is required.

## Scaling

The app is stateless aside from WebSocket connections. Vertical scaling (bigger instance) is the first lever. Horizontal scaling requires a pub/sub backend for the WebSocket broker (see Chapter 38). Database scaling is done through Railway's UI — bigger plans bump CPU, memory, and storage.

## Related chapters

- **Chapter 41** — Cache Busting and Redeployment
- **Chapter 42** — Monitoring and Health Checks
- **Chapter 43** — Database Backup and Volume
- **Chapter 44** — Incident Response

# Cache Busting and Redeployment

## In this chapter

The `.railway-cache-bust` sentinel and service-worker cache versions.

## Overview

The cache-bust strategy ensures users always run a consistent frontend version and don't get stuck on a stale deploy. It is a coordination of three layers: Vite's hashed filenames, the service worker's update flow, and a version-check hook in the React app.

## Layer 1: hashed static assets

Every JS and CSS file emitted by Vite has a content-hash in its filename (e.g., `main.a8f3e2c1.js`). Changing the source produces a new hash, so browsers see a new URL and skip any cached copy of the old one. The hashed files are served with `Cache-Control: public, max-age=31536000, immutable`.

## Layer 2: HTML shell revalidation

The entry HTML file (`index.html`) is never hashed. It is served with `Cache-Control: no-cache` so the browser always revalidates it. Since the shell references the hashed JS and CSS, an updated shell points at the new bundles automatically.

## Layer 3: service worker update

The service worker is generated at build time with a version string. A new build produces a new service-worker script. Workbox's default update flow is: new service worker installs in the background, waits for existing clients to close, then activates. To shorten that gap, the app sends `{type:'SKIP_WAITING'}` to the new worker as soon as the user navigates and triggers a one-time page reload.

## Layer 4: in-app version check

A React hook polls `GET /api/version` every 5 minutes. If the returned build SHA differs from the one the client booted with, a banner appears: "A new version is available. Reload now." Users can dismiss and keep working, but the app will not push new features to a client that refuses to reload.

## Edge cases

- Intermediate CDN caches: we do not use a third-party CDN; the hosting platform serves assets directly.
- Buggy service worker: in an emergency, an admin can trigger a "clear cache" broadcast that posts a message to all clients telling them to unregister the worker and reload.
- Browser back/forward cache (bfcache): we use `pageshow` events to trigger a version check when a restored page is shown.

## Related chapters

- **Chapter 40** — Deployment to Railway
- **Chapter 36** — Progressive Web App
- **Chapter 44** — Incident Response

# Monitoring and Health Checks

**In this chapter**

Health checks, additional status endpoints, and observability.

## Overview

Health Monitoring is the operational visibility surface: liveness and readiness probes, integration health, queue depth, error rates, and real-time WebSocket connection count. Admins keep it open on a side monitor during incidents.

## Liveness / readiness

| Endpoint         | Purpose                                                                      |
|------------------|------------------------------------------------------------------------------|
| GET /api/health  | Simple liveness — returns 200 if the process is up.                          |
| GET /api/ready   | Readiness — 200 only if DB and cache are reachable. Load balancer uses this. |
| GET /api/version | Returns the current build git SHA and deploy timestamp.                      |

## Dashboards

- Error rate: rolling 5-minute count of 5xx responses.

- P95 latency: per-route, with a badge on any route above 500 ms.
- WebSocket connections: live count, with a graph of the last hour.
- Queue depth: background-job queue length (webhooks, emails, analytics rollups).
- Integration health: Mojo, Bonzo, webhook sink, push service.

## Alerting

Alerts are emitted to #alerts in Chat and optionally to a configured PagerDuty integration. Alert rules are thresholds on the dashboard metrics above. The thresholds are conservative by default — tune them as operational baseline becomes clear.

## On-call runbook entry

If /api/ready returns non-200 for more than 60 seconds, the platform provider restarts the process. If restarts do not resolve it, page on-call. See the incident response chapter (44) for full procedures.

## Related chapters

- **Chapter 40** — Deployment to Railway
- **Chapter 44** — Incident Response

# Database Backup and Volume

**In this chapter**

The /data volume, 500 MB allocation, backups, and restores.

## Overview

CLR Connection Center runs a PostgreSQL database and a small volume of attachment files. Both must be backed up. This chapter covers the backup strategy, restore procedure, and retention policy.

## What is backed up

| Asset          | Strategy                                                                           |
|----------------|------------------------------------------------------------------------------------|
| PostgreSQL     | Nightly pg_dump to object storage. 30-day retention.                               |
| PostgreSQL WAL | Continuous WAL archiving for point-in-time recovery within 7 days.                 |
| Uploaded files | Object storage is already durable; a daily snapshot duplicates to a second region. |
| App config     | Infra-as-code repo in Git; no runtime state to back up.                            |

## Restore procedure

- 1. Provision a new Postgres instance of the same version.
- 2. Restore the most recent nightly dump: `pg_restore -d $NEW_DB latest.dump`.
- 3. Apply WAL segments up to the desired recovery target.
- 4. Point the app's `DATABASE_URL` at the new instance and redeploy.
- 5. Smoke-test: login, load dashboard, log a dummy outcome, delete it.

## Testing

A quarterly restore drill is mandatory. A staging environment is spun up from the most recent backup and a smoke-test suite runs. Results are logged in the forum under `#ops` `#restore-drill`. A drill that fails is a Sev2 incident until resolved.

## Retention and compliance

Backups are encrypted at rest with AES-256. Keys are managed in the platform's secret manager. Retention is 30 days; longer retention for any legal hold is case-by-case via Super Admin. Deleted user data is purged from nightly backups after the 30-day window, satisfying right-to-deletion requests.

## Related chapters

- **Chapter 40** — Deployment to Railway
- **Chapter 44** — Incident Response

# Incident Response

**In this chapter**

Playbooks for stuck deploys, stale caches, bad deploys, and lost passwords.

## Overview

When something breaks, the priority is: 1) restore service, 2) communicate, 3) learn. This chapter documents how on-call engineers should respond to a production incident.

## Severity levels

| Sev  | Criteria                                       | Response SLA                           |
|------|------------------------------------------------|----------------------------------------|
| Sev1 | App is down or critical data is at risk.       | Page immediately. 15-min response.     |
| Sev2 | Significant feature broken; workaround exists. | 1-hour response during business hours. |
| Sev3 | Minor feature broken or cosmetic issue.        | Next business day.                     |
| Sev4 | Tracked as a bug; no immediate action.         | Backlog.                               |

## Response playbook

- Acknowledge the page. Post "taking this" in #incidents.

- Create an incident doc from the template.
- Assess severity and set a status banner in the app for Sev1/Sev2.
- Pursue the shortest path to restore service — revert a deploy, scale up, flip a feature flag.
- Communicate every 30 minutes in #incidents until resolved.
- Post a summary once resolved. Schedule a postmortem for Sev1 and Sev2.

## Common incidents

| Symptom                                | Likely cause                    | First step                                       |
|----------------------------------------|---------------------------------|--------------------------------------------------|
| 5xx spike after deploy                 | Regression in new build.        | Revert to previous release.                      |
| WebSocket connections not establishing | Proxy misconfiguration.         | Check platform logs for upgrade errors.          |
| DB CPU at 100%                         | Runaway query or missing index. | Check pg_stat_activity; consider read-only mode. |
| Webhook receiver 429-ing us            | Burst from recalculation.       | Pause destination; contact receiver.             |
| Push notifications silent              | VAPID key mismatch.             | Verify key in .env; rotate if leaked.            |

## Postmortem

Every Sev1 and Sev2 gets a blameless postmortem within a week. Use the template in the forum under #ops #postmortem-template. The purpose is to surface contributing factors, not to assign blame. Action items go into the tracker with named owners and due dates.

## Communication templates

For user-facing incidents, post a banner via **Super Admin Status banner**. The banner appears on every page. Draft copy: "We're investigating degraded performance affecting outcome submission. Updates every 30 min." Never speculate about cause in a user-facing banner; stick to observable symptoms.

## Related chapters

- **Chapter 40** — Deployment to Railway
- **Chapter 41** — Cache Busting and Redeployment
- **Chapter 42** — Monitoring and Health Checks
- **Chapter 43** — Database Backup and Volume

# Changelog (April 2026)

## In this chapter

The April 22–23, 2026 commit window that shipped v1.0.

## Overview

This chapter tracks meaningful changes to the application since the first internal release. Entries are reverse-chronological. Each entry includes the date, a short description, and the area of the app affected.

## Versioning policy

The application is not semantically versioned; instead, every deploy produces an immutable build identified by its git commit SHA. Significant changes are tagged with "v-like" milestones (v1.0 = first external documentation, v1.1 = first integration, etc.) tracked manually in this chapter.

## Deprecations

Breaking API changes are announced in this chapter with a sunset date at least 60 days out. The server carries a X-CLR-Deprecation header on the affected endpoint during the deprecation window to give integrators a runtime signal.

## Changes

**Apr 23, 2026** 4a1f3c3c

Appointment badge filters out transfers and overdue items.

**Apr 23, 2026** 2f2003e8

Integrations item restored in the Integrations section.

**Apr 23, 2026** 68c52549

Auto-deploy verification.

**Apr 23, 2026** 100f6210

Bumped Railway cache-bust to force a fresh build.

**Apr 23, 2026** 6b22849c

Integrations moved to Help; 3-day appointment window introduced.

**Apr 23, 2026** f2f3d485

Removed duplicate Integrations link inside the Integrations section.

**Apr 23, 2026** aaa737f3

Complete EOD PDF export with a dedicated print sheet.

**Apr 23, 2026** 116eacdc

Full PWA setup with proper W-mark icons.

**Apr 23, 2026** c1cfd307

1D / 1W / 1M / All Time toggle on activity trend charts.

**Apr 23, 2026** 2e09b79e

Replaced intro video with final 5:56 version.

**Apr 23, 2026** 76b04cb9

Removed duplicate Integrations nav item from TOOLS section.

## Before v1.0

The app started as an internal tool built during Q4 2025 by a small team. The initial scope was outcome logging and EOD submission. Integration with Mojo followed shortly thereafter, Bonzo came in early Q1 2026, chat and forum in February 2026, and the PWA surface in March 2026. The v1.0 milestone represents the first complete feature set with documentation, admin tooling, and an operational playbook.

## Roadmap hints

This is forward-looking and subject to change. Probable near-term work: a mobile-native wrapper, deeper Bonzo bidirectional sync, a call recording review surface, and role-based permissions that are more granular than the current flat model. None of these are promised.

## Related chapters

- **Chapter 46** — Glossary of Terms
- **Chapter 47** — FAQ

# Glossary of Terms

## In this chapter

Definitions of CLR, PWA, EOD, VAPID, and every other term used.

## Overview

A quick-reference glossary for terms that recur through this manual. In-app there is also the admin-editable Glossary (Chapter 16) for borrower-facing terminology; this chapter is for app-specific concepts.

## Terms

**Assignment.** One LO assigned to one CLR for one calendar day. Generated once per day by the algorithm, then locked.

**Audit Log.** Chronological record of admin-level actions — assignment unlocks, role changes, LO archives, impersonations, webhook-setting changes, and hotpatches.

**Badge.** A small count displayed next to a sidebar item. Appointments badge counts active appointment-type outcomes due in the next three days. Chat badge counts unread messages. NMLS badge counts pending license checks.

**Bonzo.** A CRM used by West Capital Lending to track prospects through loan pipelines. Integrated via webhook and a full sync endpoint.

**Cache Bust.** The act of bumping `.railway-cache-bust`` (a plain timestamp file) to force Railway to skip its build cache and rebuild from scratch.

**Callback Requested.** Outcome type meaning the borrower asked to be called back later. Has a follow-up date and appears on the Appointments page until resolved.

**CLR.** Client Loan Representative. The primary user role — makes outbound calls to borrowers, logs outcomes, files EOD reports.

**Contact Hub.** A unified contact view blending entries from Mojo, Bonzo, and manual imports into a single searchable directory.

**DNC.** Do-Not-Call. Contacts flagged in Mojo or the internal DNC list are excluded from calling lists.

**EOD Report.** End-of-Day report. Each CLR submits one per calendar day summarizing calls made, outcomes, LO coverage, and notes.

**Fell Through.** Outcome meaning an expected transfer did not complete — borrower didn't pick up the warm handoff, LO couldn't take it, etc.

**Follow-Up Date.** Date stored on an outcome to indicate when the borrower should be contacted again.

**Forum.** Long-form Q&A; board with posts, threaded answers, upvotes, and accepted-answer marking.

**Future Contact.** Outcome indicating a contact should be re-attempted after a meaningful interval (weeks / months).

**Hotpatch.** Admin-only endpoint (`POST /api/admin/hotpatch``) used in emergencies to apply minor server-side changes without a full redeploy. Audited.

**LO.** Loan Officer. Each CLR works through a rotating list of assigned LOs.

**LO Vacation / Snooze.** Flag marking an LO as temporarily unavailable. Excluded from daily assignment generation.

**Maskable Icon.** A PWA icon variant whose important content sits inside a safe zone so Android can crop it into circles, rounded-squares, etc. without cutting off the W-mark.

**Mojo.** The outbound dialer West Capital Lending uses. Sessions (call volume, DNCs, transfers) are ingested by webhook and optional CSV import.

**NMLS.** Nationwide Multistate Licensing System. Used to verify loan officer license validity.

**No Answer.** Outcome type — borrower did not pick up, no voicemail reached or left.

**Parse DB Timestamp.** Frontend helper that converts SQLite's UTC-looking timestamp strings (no `Z`) into proper local `Date` objects.

**PWA.** Progressive Web App. Installable version of WCLCC for iOS, Android, and desktop with offline asset caching and push notifications.

**Service Worker.** The `wclcc-v2` service worker that pre-caches the app shell, icons, and critical assets for instant loads and offline support.

**Session.** Cookie-based authentication record. Created by `POST /api/auth/login`, destroyed by `POST /api/auth/logout`.

**Super Admin.** Role that can view and manage multiple organizations, invite admins to each, and impersonate into any org for support.

**Transfer.** Outcome meaning a CLR successfully handed a prospect off to an LO. Two sub-types: direct transfer and appointment transfer.

**VAPID.** Voluntary Application Server Identification — the key pair used to sign Web Push notifications. Public key exposed at `GET /api/push/vapid-public-key`.

**W-mark.** The WCL logo — the embossed gold `W` in a circle used across the PWA icon set and cover pages.

**Webhook.** Inbound HTTP endpoint under `/api/webhook/\*` that accepts events from Mojo, Bonzo, or Zapier. Verified by a per-integration shared secret.

**wlc.it.com.** Public domain bound to the Railway deployment. Branded `WCLCC` in app chrome.

## Related

See Chapter 16 for the borrower-facing Glossary surface managed by admins. See Chapter 47 for frequently-asked questions.

## Related chapters

- **Chapter 16** — Glossary
- **Chapter 47** — FAQ

# FAQ

## In this chapter

Expanded FAQ covering daily operations, admin, and deployment.

## Overview

Questions that come up often from CLRs, admins, and new engineers. If you don't see an answer here, try the forum (Chapter 13) or reach out to your admin.

### Q1. How do I generate today's calling list?

Go to Settings (admin-only) and click **Generate Daily Assignments**. Only one generation per day is allowed; subsequent clicks require an unlock with a reason and are recorded to the audit log.

### Q2. Can I change my assignment after it's locked?

Only an admin can unlock and regenerate with a reason. The action is audited. Per-row overrides are also available from the Assignments page under admin view.

### Q3. What if an LO is missing from my list?

The LO may be archived or marked unavailable. Check **Directory** and **LO Vacation**. Unavailable LOs are filtered out at generation time.

### Q4. How do I submit my EOD report?

Open **EOD Report** from the sidebar. Transfers and appointments auto-populate from today's outcomes. Fill out notes, assigned-called list, and additional LOs, then click **Submit**. Today's draft auto-saves as you type.

#### **Q5. How do I print or export an EOD report?**

From the EOD history list, open a past submitted report. Use the **Print / Export PDF** action. The export produces a dedicated print sheet with identity, outcome breakdown, transfer prospects, LO coverage, notes, activity log, and signature lines — not a screenshot of the form. Available for past reports only.

#### **Q6. How do I install the app on my phone?**

Go to **Help Install App** in the sidebar. iOS: Safari Share Add to Home Screen. Android / desktop Chrome: the browser prompts, or use the install icon in the URL bar.

#### **Q7. Why did my appointment badge count change?**

As of April 23, 2026, the badge only counts active appointment-type outcomes with a follow-up date today through three days out. Transfers and overdue items are excluded — if your badge dropped, it's because old overdue rows used to be counted and now aren't.

#### **Q8. How do I change the date range on the activity chart?**

Use the **1D / 1W / 1M / All Time** toggle above the chart. Your selection persists per-device via localStorage. Default is 1M.

#### **Q9. Why does my script auto-fill some words?**

Placeholders (e.g. [your name], [lo name], [borrower name]) are resolved in real time from your profile and the currently selected LO / borrower. Admins can edit the default script or any CLR's personal copy.

#### **Q10. What happens when I mark an LO on vacation?**

The LO is excluded from daily assignment generation until you un-snooze. Historical call data is preserved. Assignments already generated before the snooze are not retroactively removed.

#### **Q11. How do I reach someone for help?**

Primary: Ethan Wood — ethan.anthony.wood@gmail.com. Secondary: Chris Redoble. Or use the Forum for team-visible questions.

#### **Q12. How do I reset my password?**

On the login screen click **Forgot password?** and enter your email. A reset link is emailed to you; it expires after a short TTL. While logged in, use **Settings Change Password**.

### **Q13. What's the difference between Mojo and Bonzo?**

Mojo is the outbound *dialer* — it powers the actual calls and reports call volume, DNCs, transfers. Bonzo is the *CRM* — it stores prospects with pipeline stages, long-term notes, and drip messaging. Both feed the Contact Hub.

### **Q14. Where do the numbers on the leaderboard come from?**

Directly from call outcomes and EOD reports. Outcomes are authoritative for calls, transfers, appointments, fell-throughs, callbacks and future-contacts. EOD reports add narrative, LO coverage, and manual additions.

### **Q15. What are the daily assignment algorithm weights?**

Recency (days since last called), frequency (how often this LO's been worked), availability, transfer rate, and priority tier. Admins can tune the weights via **Settings Algorithm** (backed by GET/PATCH /api/settings/algorithm).

### **Q16. How big can the database grow on Railway?**

The mounted volume is 500 MB. Current usage is ~52 MB. Outcomes, chat, and audit-log are the largest tables — rotate via the export endpoints under /api/export/\* if needed.

### **Q17. Does the app work offline?**

The service worker (wclcc-v2) pre-caches the app shell, icons, and critical assets, so the UI loads offline. API calls still require connectivity. Log entries attempted while offline fail visibly — no silent write-and-sync.

### **Q18. Can I get Web Push notifications?**

Yes, on the PWA. Allow notifications in your browser. The app subscribes via VAPID (GET /api/push/vapid-public-key). Toggle in **Settings Notifications**.

### **Q19. Where's the audit trail for admin actions?**

**Audit Log** in the sidebar (admin only). Sources: GET /api/audit-logs. Captured actions: assignment unlocks, user role changes, LO archive, hotpatches, impersonation, webhook-secret rotation.

### **Q20. How do I spin up a second organization?**

Only super admins can. Go to **Super Admin Organizations** and click **New Org**. Invite the first admin from there. Backed by POST /api/super-admin/orgs and POST /api/orgs/:id/invite.

## Adding to this list

If you find yourself answering the same question twice, file a forum thread with the answer and ping an admin to add it here in the next manual revision.

## Related chapters

- **Chapter 45** — Changelog (April 2026)
- **Chapter 46** — Glossary of Terms
- **Chapter 48** — Contact and Support

# Contact and Support

## In this chapter

Primary and backup support contacts, self-service, and bug reporting.

## Overview

If something is broken, someone is ready to help. This chapter lists the escalation paths in order of preference.

## Inside the app

- Help button (lower-left) opens the in-app support panel with article search.
- Chat #general for non-urgent questions and announcements.
- Chat #incidents for anything that looks like an outage (admin-pinned).
- Forum for durable questions that should stay findable.

## Outside the app

- Email ops@example.com for operational issues outside business hours.
- On-call phone tree for Sev1 incidents — accessible from **Super Admin Contacts**.
- Slack bridge (if enabled) for organizations that prefer Slack over in-app chat.

## Providing good detail

The fastest way to get a bug fixed: reproduction steps, what you expected, what you saw, screenshot or screen recording, the URL, the browser + OS, and the time of the incident. The in-app support form pre-fills most of this.

## Service expectations

| Severity        | Response SLA      | Resolution target |
|-----------------|-------------------|-------------------|
| Sev1 (down)     | 15 minutes, 24x7  | 4 hours           |
| Sev2 (partial)  | 1 hour (business) | 1 business day    |
| Sev3 (minor)    | 1 business day    | 1 week            |
| Sev4 (cosmetic) | Best-effort       | Next release      |

## Feedback

Product feedback — "I wish the app did X" — goes to the forum under #feedback. Good feedback includes the problem you're trying to solve, not just the solution you want. Popular feedback bubbles up in monthly product review.

## Closing

Thank you for using CLR Connection Center. This manual has covered every feature, page, admin surface, integration, API endpoint, and operational procedure that makes up the v1.0 application. If you notice something missing, file a documentation issue and it will be addressed in the next revision.

## Related chapters

- **Chapter 45** — Changelog (April 2026)
- **Chapter 46** — Glossary of Terms
- **Chapter 47** — FAQ